

Front matter (booklet v1, 2026-10-09)

Title. Grid vs. Graph Representations for Self-Play Learning in Hive: a Pre-Registered Comparison under Limited Compute

(Descriptive, no presumed result, per plan ch. 20.)

Author. Mohamed Bechir Kefi

Status. Independent research report — not peer-reviewed, not a publication of any institution. Version 1.0-draft, 2026-10-09.

Code and artifacts. Repository `hive-graph-selfplay` (release tag and access to be fixed at diffusion time, G-PUBLIC); results manifest under `results/`; every figure and table regenerates from scripts over raw per-game records.

AI assistance declaration. This study was executed under a human-as-PI methodology in which an AI research assistant (Claude, Anthropic) implemented code, ran campaigns, and drafted text under a gate system reserving all scientific decisions — freezes, budgets, spending, publication — to the human author, who owns every claim. Chapter 8 documents this working methodology in full; the source document lives in the repository (`docs/methodology.md`).

Abstract (164 words). Hive is a boardless hexagonal strategy game whose moves are (piece, destination) pairs over an ever-changing set of cells — a natural candidate, in principle, for graph neural encodings over the convolutional grid encodings standard in AlphaZero-style systems. We test that intuition under a pre-registered protocol frozen before any comparison run: one grid CNN and one capacity-matched (+1.5%) relational message-passing network share a rules-validated engine, one action decoder, identical training settings, a frozen three-opponent population and 250 frozen openings, evaluated under two budget readings (equal training examples; equal wall-clock at a pre-registered cutoff) with five independent seeds per arm and truncation reported as its own outcome. The hypothesis is rejected: the graph arm scores lower against two of three opponents under both readings, at twice the wall-clock cost, with its deficit concentrated in converting won positions. Ablations show the graph arm's typed edge relations are load-bearing for optimization stability, while its global pooling is dispensable. Negative, pre-registered, and fully reproducible from released records.

Keywords. Hive; AlphaZero; graph neural networks; representation learning; pre-registration; negative result

Table of contents. 1 Introduction · 2 Formalisation · 3 Related work · 4 Method (engine validation; baselines; pipeline; representations) · 5 Protocol · 6 Results (comparison; ablations) · 7 Discussion and threats · 8 Working methodology (gated human–AI) · 9 Conclusion · Bibliography · Webographie · Annex A Position corpora · Annex B Architectures, decoder, formats · Annex C Hyperparameters, seeds, commands · Annex D Repository pointers · Annex E Result tables and figures (generated)

Chapter 1 — Introduction (booklet draft, 2026-10-09)

Written last-but-abstract per the plan's order, from final results.

The question. Hive is a boardless hexagonal strategy game: pieces define the playing surface, stacks form and dissolve, and every move is a (piece, destination) pair over a set of cells that changes each turn. Grid-based encodings — the convolutional lingua franca of AlphaZero-style systems — must bolt a frame onto this frameless game. A graph encoding needs no frame at all. It is natural to expect the graph to win. Does it, at a compute budget a single machine can afford?

Why it is not obvious. The intuition cuts both ways. Graph networks match the game's native structure and carry no anchoring artifacts; but Hive's outcomes hinge on short-range surround tactics — the regime in which convolutional locality is strongest — and prior evidence splits: graph arms won in Hex under DQN (Keller et al. 2023) and in chess under self-play (Rigaux & Kashima 2024, from single runs), while the only AlphaZero-on-Hive study (de Goede et al. 2022) never tried a graph at all. The hypothesis was stated falsifiably and frozen before any comparison run, together with everything that could bend the answer: opponents, openings, budgets, evaluation settings, and the rejection rule itself.

What we did. We built both encodings behind one shared action decoder on one rules-validated engine, matched capacity to +1.5%, and trained five independent seeds per arm under identical self-play settings, reading the comparison two ways — equal training examples and equal wall-clock — against a frozen three-opponent population on 250 frozen openings, with truncation as a first-class outcome throughout.

What we found. The hypothesis is rejected. The grid arm scores higher against two of the three opponents under both readings (largest interval upper bound for a graph advantage: +0.028), at half the wall-clock cost; the graph arm's deficit concentrates in local tactical conversion — it wins material against the random opponent and then fails to close, truncating 20–57% of those games where the grid arm truncates almost none. Ablations localise the graph arm's machinery: its direction-typed edge relations are load-bearing for optimization itself (removing them diverges training in every seed), while its global pooling is dispensable.

Contributions. (1) The first controlled, budget-matched, multi-seed grid-vs-graph comparison for Hive, both arms under the same AlphaZero-style pipeline, with a pre-registered negative answer — Chapter 6 and [results/comparison/](#). (2) A reproducible comparison harness for frameless, stacking games: a validated rules engine with cross-language golden-pinned encoders, a shared variable-action decoder, truncation-aware evaluation machinery, and frozen-artifact discipline — Chapters 4–5 and the released code. (3) A component attribution for the graph arm (edge typing = trainability; pooling = null) with an honestly labeled supplement — Chapter 6 (H-T3) and [results/ablations/](#).

Each contribution is verifiable from the repository: every number in this booklet traces to a journal entry and regenerates from scripts over raw per-game records.

Chapter 2 — Formalisation (booklet draft, 2026-09-26)

Sources: *frozen protocol v1.0 (D-020)*; *docs/action-decoder.md (D-015)*; *docs/representations/*; *engine architecture notes*. No numbers here beyond registered ones.

2.1 The game and the studied variant

Hive is a two-player, perfect-information, zero-sum game played on an unbounded hexagonal tiling: there is no board — the pieces in play define the playing surface. We study the **base game only** (queen, spiders, beetles, grasshoppers, ants; 11 pieces per side), decision D-007. The engine implements the Mosquito/Ladybug/Pillbug expansions and the tournament opening restriction (no queen on a player's first turn); the kernel is exercised by tests across all eight game types, but no study result involves an expansion.

2.2 State, actions, transition, result

A state s comprises: the placed pieces with their cells and stack levels (beetles may climb, producing stacks), the reserves, the side to move, the ply count, and the stun marker `last_moved` (relevant only under the Pillbug expansion; carried for kernel uniformity — it is part of the hashed position). The engine exposes s through the Universal Hive Protocol (UHP); its rules correctness is established independently of any learning component (Chapter 4).

An **action** is (piece, destination cell), which identifies any Hive move uniquely — walk-vs-throw collisions produce identical successor states — plus a distinguished *pass*, legal exactly when no move exists. The legal-action set $A(s)$ is produced by the engine's generator; both learned architectures receive it and normalise their policies over exactly $A(s)$ (the shared decoder, Chapter 4). Transitions are the engine's `play`; the game ends when a queen is fully surrounded (win for the opponent; simultaneous surround = draw), with additional draw by threefold repetition.

Official result vs experimental truncation. Self-play and evaluation games are stopped at a 300-ply cap. A capped game is **not** a draw: it is a fourth outcome class, *truncated*, carried through the data format (records), the training loss (truncated games are excluded from the value target), the match runner, all tables (separate rate column) and the analysis. The reward for learning is the terminal result from the side-to-move perspective: win +1, loss -1, draw 0; truncation contributes no value target.

2.3 Search and learning notation

Both arms use the same PUCT Monte-Carlo tree search: at a node, an evaluator returns priors over $A(s)$ and a scalar value $v \in [-1, 1]$ (side-to-move perspective); selection maximises $Q + c \cdot \text{prior} \cdot \sqrt{N}/(1+n)$ with $c = 1.4$; terminal values back up exactly. Self-play uses playout-cap randomization (a fraction 0.25 of decisions get 128 simulations and are recorded; the rest get 32 and are not), Dirichlet root noise ($\epsilon = 0.25$), temperature sampling for the first 12 plies, and resignation below -0.92 with a 10% no-resign audit fraction. **Evaluation applies none of the exploration machinery** ($\epsilon = 0$, deterministic argmax; enforced by test and pinned by config, D-019) and runs 400 simulations per decision.

Training minimises a policy cross-entropy against the recorded MCTS visit distributions, normalised over the legal set for both arms, plus a weighted 3-class value loss (weight 0.6) over win/draw/loss with truncated samples excluded. Budgets are accounted in four denominations per run — wall-clock,

hardware, training states, simulations — and the comparison is read under two equalisations (same-examples, same-wall-clock), Chapter 5.

2.4 Game view vs encoding view

The same state s is encoded two ways (figure fig3-encodings): the *grid view* embeds the position in a fixed 32×32 frame (BFS-unwrapped from the engine's toroidal board, bounding-box-centred) with 77 feature planes; the *graph view* is coordinate-free — nodes are the occupied cells plus every empty cell adjacent to the hive (exactly the decoder's destination universe), edges carry the six hex directions as types, stacks appear level-by-level as node features, and reserves/side/ply enter as a global vector. Chapter 4 specifies both; nothing else in the system differs between the arms.

Chapter 3 — Related work and positioning (booklet draft, 2026-09-26)

Sources: `docs/reading/` (verified notes; one row per work in `docs/reading/matrix.md`); decision D-009 (scoped claim). Only noted, actually-consulted sources are cited; full metadata lives in the `bibliography/webographie`.

3.1 Self-play reinforcement learning at scale — and small scale

AlphaZero (Silver et al. 2017/2018) fixed the algorithmic template this study inherits: a policy-value network guiding PUCT-MCTS self-play, with the state presented as stacked spatial planes ($8 \times 8 \times 119$ for chess) and the policy itself expressed spatially — the canonical grid parameterisation our grid arm descends from. Its evidence style, however, is what a small-compute study must *depart* from: a single training run per game, budgets denominated in hardware (5,000 first-generation TPUs) rather than in comparable units, and evaluation against one reference engine. KataGo (Wu 2020) showed the pipeline’s cost is compressible by roughly $50\times$ at equal strength (under 30 V100s for 19 days versus ELF OpenGo’s ≈ 74 GPU-years) and set the budget-reporting standard — GPU-days, games, samples, with per-technique ablations — that our accounting follows; of its economies we adopt exactly one, playout-cap randomization, because it is representation-agnostic and can be applied identically to both arms, and we deliberately skip the Go-specific input features and auxiliary targets that would smuggle domain knowledge into one encoding. Jones (2021) trained AlphaZero-style agents across Hex board sizes on ≈ 500 GPU-hours total and found smooth compute-performance frontiers — the closest methodological licence for drawing conclusions at our scale — and contributed a warning we encode in the protocol: train-time and test-time compute trade off, so evaluation visit counts must be pinned, not floated. Agarwal et al. (2021) supply the statistical frame: in few-run regimes, point estimates over single runs frequently reverse under proper interval analysis; their prescriptions — aggregate per run first, resample the run, report intervals, never pool games as independent observations — are implemented here with the seed as the resampling unit throughout.

3.2 Graph representations and variable action spaces

GraphSAGE (Hamilton et al. 2017) grounds the inductive message-passing family our graph arm belongs to; vanilla GraphSAGE lacks edge semantics, which our direction-typed relations add. Pointer Networks (Vinyals et al. 2015) justify scoring a variable candidate set — our shared decoder scores exactly the legal (piece, destination) pairs, with empty destination cells as first-class scorable objects. MDP-homomorphic networks (van der Pol et al. 2020) motivate the *time-permitting* symmetry question only; per the protocol, no invariance is assumed of a GNN, and none was claimed.

3.3 Hive and grid-vs-graph comparisons

Scholarly Hive AI is thin. Kampert et al. (2021) built heuristic minimax/MCTS agents on the BeeKeeper engine, documented Hive’s ≈ 60 branching factor, and found that an intuitively central feature (tiles around the queen) carries surprisingly little evaluation signal once tuned — an early warning that Hive’s value structure is not where intuition puts it; their agents, like every scholarly Hive agent before ours, remained below strong human play. AZ-Hive (de Goede et al. 2022) is the closest prior work and the study’s direct motivation: AlphaZero on Hive across a 5×2 design space of board and action encodings — all dense hex-lattice arrays into a CNN, no graph option anywhere in the space (confirmed against the full text) — with the stark result that after 24 h of training their best engine still lost to plain MCTS and

minimax (BayesElo 1063 vs 1181/1355), while encoding choice measurably changed early learning speed. That is precisely RQ-H1’s premise: in Hive, the encoding is load-bearing. Polygames (Cazenave et al. 2020) achieves boardsize invariance — fully convolutional bodies with global pooling — but within the grid paradigm and without Hive support: it scales fixed-topology boards, which a boardless, stacking game does not offer. The direct grid-vs-graph precedents are two. Keller et al. (2023) ran a parameter-matched CNN-vs-GNN comparison on Hex ($\approx 487\text{K}$ vs $\approx 481\text{K}$ parameters, ≈ 110 A100-hours per model) and found the asymmetry our result extends: the GNN dominated long-range-dependency tests and transferred across board sizes, while the CNN remained sharper at local patterns — but their controlled comparison ran under RainbowDQN, their CNN arm never trained under MCTS self-play, their graph is a Hex-specific Shannon-game reduction (played cells are contracted away; actions biject to nodes), and they state themselves that the construction does not generalise to other games. Rigaux & Kashima (2024, NeurIPS) report the opposite sign for chess: an edge-featured graph-attention network (GATEAU) with an edge-based policy readout out-learns CNN baselines in an AlphaZero-style loop and transfers across board sizes — but from a single training run per model (intervals cover Elo estimation only, not run variance), with loose capacity matching (1.0M vs 2.2M parameters) and a decoder that differs between arms, confounding representation with action parameterisation; our design removes exactly those three confounds. Ben-Assayag & El-Yaniv (2021) trained a GIN-based AlphaZero on Othello lattice graphs to scale small-board training to larger boards — a transfer claim under deliberately asymmetric budgets, not an equal-budget representation comparison, though notably with the best replication hygiene of the three (five runs with standard errors). An unpublished hobby project (hiveGo; webographie) trains a small GNN on Hive with an AlphaZero-style loop and reports only anecdotal evaluation — no controlled comparison of any kind.

3.4 Positioning

No prior work runs a controlled, budget-matched grid-vs-graph comparison for Hive with both arms under the same self-play pipeline; that is this study’s scoped contribution (D-009) — deliberately *not* “the first grid-vs-graph comparison in a board game”, which Keller et al. and Rigaux & Kashima preclude. The scope closes as follows: one variant (base Hive), one grid net and one simple relational GNN at matched capacity, one machine, frozen opponents/openings/protocol, two budget readings, five seeds per arm. Within that perimeter the comparison is answered (Chapter 6); outside it, nothing is claimed. The contrast with Rigaux & Kashima’s positive chess result and Keller et al.’s long-range-vs-local asymmetry is taken up in the discussion.

Method — Engine correctness (report section draft)

Drafted 2026-09-09 (invariant 14: written as the phase runs). English working master; French copy at the report milestone (D-006). Every number traces to a journal entry (invariant 4); claims are registered in `claims.md`.

Why validation precedes everything

A self-play study trains on positions the engine itself generates and scores; a rules defect therefore does not merely add noise — it teaches both arms a systematically wrong game and silently invalidates the comparison (“false learning”, plan ch. 4). The stop rule is absolute: an engine found incorrect suspends all training-side work (invariant 11). We therefore validate along four independent lines before any baseline or training work, and report each line with its exact scope.

Four lines of evidence

1. Perfth against published tables. Node counts for all 8 game types match Mzinga’s published perfth tables: depth ≤ 5 in the standard test suite, depth 6 in a dedicated run (8.3 s, all types), depth 7 in the nightly script. Perfth is exhaustive within its depth: any movegen discrepancy — a missing move, an extra move, a wrong stacking rule — shifts a count. (Journal H2-2026-09-09-suite-rerun-01.)

2. Reference-engine agreement. The engine passes nokamute’s UHP conformance harness (21/21) and, more stringently, differential fuzzing plays seeded random games while asserting per-ply *set equality* of legal moves against two independent reference engines — MzingaEngine v0.16.0 (the UHP reference implementation) and nokamute 1.0.3: 27,829 positions at the session’s fixed seed, 200/100 games per type nightly. Two engines with independent codebases agreeing on every legal-move set bounds the chance of a shared rules misreading. (Journal H2-2026-09-09-suite-rerun-01.)

3. Hand-annotated critical corpus. Reference agreement cannot catch a misreading shared by the community’s engines, so 30 critical positions were annotated *by hand from the publisher’s rules* (Gen42 rulesheet and Pillbug sheet, World Hive Tournaments FAQ; tournament opening rule declared as a convention) — sliding and gates, beetle stacking and the above-ground beetle gate, stack colour for placement, One-Hive cut points and rings, win/draw termination including the simultaneous-surround draw, forced pass, and Pillbug stun as a kernel guard. Expectations were committed before the first engine run; the runner compares full (piece, destination) move sets over UHP. First run: 29/30, with the single disagreement resolved *against the corpus* — a setup line violated the One-Hive transit rule the engine correctly enforces — and 30/30 after the correction; the investigation is journaled either way (invariant 2). Declared limit: the annotations have not yet had an external Hive-literate review. (Journal H2-2026-09-09-corpus-run-01.)

4. Random invariant sessions. Seeded random games across all 8 game types check, after every transition: every generated move is accepted by the apply path (and undo restores the exact GameString), the hive remains connected, serialise→deserialise through the UHP GameString reproduces the position, result, and exact valid-move set, and pass is accepted exactly when no move exists. 10,665,686 generated moves were applied and undone over 161,546 plies (deep session) with zero violations; a smaller session runs in the standard suite on every build. (Journal H2-2026-09-09-random-invariants-01.)

Scope and honest gaps

Code coverage is not claimed as correctness evidence (plan ch. 4). The lines above bound different failure modes (exhaustive-at-depth, agreement, rules-text fidelity, invariant stability) but none proves full-game perfection. Known issues found during profiling and deferred to the pipeline phase, recorded as findings rather than silently fixed: the *prior-work* self-play generators map their 300-ply cap to a draw (invariant 7 forbids this in any new-study measurement code), and the Rust ONNX CoreML execution provider currently crashes on the study machine while the same model runs on CoreML through Python onnxruntime at 2.62 ms/eval. (Journal H2-2026-09-09-throughput-profile-01.)

Cross-language data-path check

The grid arm's Rust plane encoder and the Python training-side decoder are kept byte-identical, verified by a golden-file crosscheck (240 positions, exact agreement) that runs in the nightly script. (Journal H2-2026-09-09-suite-rerun-01.)

Method — Baseline opponents and evaluation population (report section draft)

Drafted 2026-09-09 at the H3 close (invariant 14). English working master; French copy at the report milestone (D-006). Numbers trace to journal H3-2026-09-09-baselines-01; claims registered in `claims.md`.

Why a frozen population

Both architectures are scored by mean result against a **fixed opponent population frozen before any training run** (protocol §4): win = 1, draw = 0.5, loss = 0, truncations excluded and reported as their own rate. A population fixed in advance is what makes scores comparable across arms, seeds and budgets; touching an opponent afterwards would silently move the measuring stick, so the population is under freeze discipline — it was frozen on 2026-09-09 with explicit approval (D-017: agent configs and weight file by sha256, engine code commit) and no member may be added, removed or retuned since. If an Elo-style number is ever quoted it is descriptive and relative to this population only.

The three agents

All three run on the validated engine (the same rules kernel as both study arms) over UHP, fully seeded:

- **B-RND — legal-random.** Uniform over legal moves; deterministic given (seed, game history). The floor and score anchor.
- **B-HEU — documented heuristic.** Greedy one-ply argmax of a handcrafted evaluation (queen-safety dominant, mobility-as-material, small tempo terms — every feature and weight tabulated in `docs/baselines.md`), plus the searcher's quiescence on enemy-queen-targeting moves. The weights were inherited unchanged from the prior engine and are pinned by an automated test: no tuning occurred during H3 and any later change breaks the suite. This closes the tune-after-observing-results path by construction.
- **B-MCTS — search without network.** PUCT MCTS with uniform priors and the same handcrafted evaluation (tanh-squashed) as leaf value; no exploration noise at evaluation. Budget: 6400 simulations per decision — chosen from measured cost (≈ 27 ms/decision single-thread on the study machine), not from the plan's placeholder figures.

Search verification before use

The MCTS baseline was verified on a hand-annotated tactical set written from the publisher's rules (same discipline as the engine-validation corpus, committed before any run): mate-in-1 by perimeter walk and by grasshopper jump, from both colours, and an avoid-self-surround case — 5/5 at 400, 1600 and 6400 simulations. Value signs under player alternation are pinned by automated tests at three levels: the evaluation negates exactly when the side to move flips; alpha-beta scores a mate-in-1 above the mate threshold for the mover, whichever colour moves; the MCTS root value is strongly positive for the winning mover, whichever colour moves.

Characterisation

100 paired, colour-swapped games per pairing from common seeded random openings, truncations reported separately (none occurred in 300 games):

Pairing	W/D/L	Score	Elo (descriptive)
B-HEU vs B-RND	100/0/0	100.0%	$\approx +2400$
B-MCTS vs B-RND	99/1/0	99.5%	+920 [+730, +1200]
B-MCTS vs B-HEU	23/29/48	37.5%	-89 [-150, -32]

One ordering surprised us and is reported as found: the 6400-sim search sits *below* the one-ply heuristic. Diagnosis (supported by a 4×-budget probe reaching 56.2%): with uniform priors, 6400 simulations spread over Hive’s ~60-move branching yield an effectively shallow search, while the greedy agent’s queen-targeting quiescence is tactically sharp — an echo of the literature, where plain search and heuristics are strong in Hive (de Goede et al. 2022; Kampert et al. 2021). No retuning followed the observation; the population deliberately spans a floor plus two mid-band opponents of different styles roughly 90 Elo apart.

Limits

Characterisation volume is the pilot’s 100 games per pairing with an unpaired-approximation interval; the tactical set (5 cases) and its annotations are unreviewed by an external Hive-literate reader — the same declared limit as the engine-validation corpus.

Method — Policy-value pipeline and pre-training checks (report section draft)

Drafted 2026-09-10 at the H4 close (invariant 14). English working master; French copy at the report milestone (D-006). Numbers trace to journals H4-2026-09-09-coreml-fix-01 and H4-2026-09-10-pilot-01; claims registered in `claims.md`.

The shared action decoder

Both architectures emit a policy over the same action space — a move is (side-relative piece slot, destination cell) plus pass — with identical legal-set masking, normalisation over exactly the legal set, identical MCTS visit-distribution targets, and a deterministic tiebreak (`docs/action-decoder.md`, D-015). The grid arm materialises this space as a 28,673-way tensor over its 32×32 frame; the graph arm (H5) scores the identical candidate pairs per position with empty destination cells as first-class nodes. Nothing about the action interface differs between arms; only the state encoder and scoring function do — this is the study’s central confounder control.

Data discipline

Self-play records are versioned and self-describing: each carries the generating model’s stamp (generation + net hash), the position, the visit-distribution target, the legal-move index list that enables masked training, and a four-valued outcome — win, draw, loss, or **truncated** (a ply-cap game is never recorded as a draw; truncated records are excluded from the value loss). A per-run manifest ties every shard to its model, and the run layout separates `selfplay/` (training inputs), `checkpoints/`, and `eval/` (never training input), enforced by an audit script.

Seven pre-training checks

All automated, run as one command against real shards before any training is trusted: (1) the masked softmax puts exactly zero mass on illegal actions and sums to one over legal actions, on real positions through a real forward pass; (2) `encode→id→decode` is the identity on all legal moves across game types (and stored targets are always legal); (3) outcomes are correct for the player to move, both colours, with truncation distinct; (4) a fixed tiny batch overfits to near-perfect fit — policy `argmax` 15/15, value 15/15, residual KL 0.09 against soft targets (the criterion is KL to the target-entropy floor: soft visit distributions have irreducible entropy, so “loss → 0” is the wrong test); (5) `save/resume` reproduces bitwise-identical model and optimizer state, with the interruption preserving the planned LR schedule; (6) exploration noise is structurally evaluation-impossible (the eval path’s defaults carry $\epsilon = 0$, asserted by test; settings pinned by config + decision D-019); (7) no evaluation game can reach training, by layout and audit.

Small-budget pilot

One full generation from a seeded random initialisation (1.44M-param grid net, 300 games at 128/32 simulations with playout-cap randomization): 17,237 positions, 56.7% of gen-0 games truncated (reported, not folded into draws), training non-degenerate (policy above chance, value above base rate, losses decreasing), and independent evaluation against the frozen population under pinned settings: **100–0 vs legal-random**, 3.3% vs the documented heuristic, 3.3% vs 6400-sim MCTS. The diagnosis is clean and follows the mandated order (rules, signs, search, data — the first three independently

validated in H2/H3): after one generation the gap to strong opponents is a data and iteration gap, and the indicated lever is more generations, not more capacity. Budget confirmation: worst-case (gen-0) generation cost is ≈ 12 s/game wall at 4 threads, falling toward ≈ 3 s/game as play sharpens with a trained net.

Limits

Single seed end-to-end at pilot scale; 30 evaluation games per opponent; only the grid arm exists yet (the graph arm arrives in H5 against the same decoder contract); gen-1+ iteration exercised structurally, not run.

Method — The two representations (report section draft)

Drafted 2026-09-10 at the H5 close (invariant 14). English working master; French copy at the report milestone (D-006). Numbers trace to journals H5-2026-09-10-encoders-01 and H5-2026-09-10-graph-wiring-01; claims registered in `claims.md`.

What varies, and what provably does not

The study's independent variable is the state representation and its network body — nothing else. Both arms share the engine, the search, the frozen opponents and pinned evaluation settings, the self-play records, the training targets and outcome conventions, the action space with its legal-set normalisation, and the training-loop machinery. Each shared element is enforced rather than asserted: the decoder is one contract with identical masking; the records are one format; each encoder is pinned by a cross-language golden test (240 positions for the grid planes, 160 for the graph tensors, byte-exact, run nightly); and both arms evaluate through the same Rust MCTS via ONNX exports.

The grid arm

The baseline representation embeds the position in a fixed 32×32 frame: BFS-unwrapped from the engine's torus, bounding-box-centred, with 77 feature planes (piece/owner/type/height jointly, pins, stun, placement regions, and global scalars as constant planes). A 28-piece hive plus its full ring of candidate destinations fits the frame by construction; that argument is enforced by an always-on assert and extremal tests (28-piece straight lines: complete, alias-free mapping) — no piece can vanish silently, in release builds included. The network is a KataGo-lite ResNet (1.44 M parameters) with a flat (piece-slot $\times$ frame-cell) policy head.

The graph arm

The graph representation is coordinate-free: nodes are the cells of the candidate set — every occupied cell and every empty 1-ring cell, so each scorable destination is a first-class node; edges are typed by the six hex directions; node features carry the stack level-by-level (owner and bug type per level), pins, stun, and placement regions; reserves, side-to-move, ply and game type enter as a global vector. Pieces are not separate nodes: the shared decoder addresses moves as (piece slot, destination cell), so piece identity enters the policy through a slot embedding and the piece's standing node — a documented design choice, not an accident. The network is a small relational message-passing net (direction-specific weights, global-pooling bias, masked pooling for the value head, per-candidate move scoring) at 1.47 M parameters — a +1.5% capacity difference, reported.

A graph network grants no invariance or rule equivalence for free. The encoding contains no absolute coordinates, but the learned function is not thereby translation- or rotation-invariant, and no rule of Hive is built in; any such claim in this study is measured, never assumed.

Measured cost asymmetries (reported, not equalised)

On the study machine, the best available inference path differs per arm: the convolutional grid net runs on the CoreML accelerator at 2.62 ms/evaluation, while the gather-heavy graph net runs fastest on CPU at 3.67 ms (CoreML is slower for it) — a $\approx 1.4 \times$ per-decision cost against the graph arm. Training shows

the reverse pattern per device (CPU favours graph 3.5×; the MPS path used for training favours grid $\approx 2\times$). These are genuine hardware interactions of the representations; the protocol's two budget readings (same-examples and same-wall-clock) charge them honestly rather than hiding them.

Symmetry augmentation

Excluded from the full method for both arms (D-023): exclusion makes the identical-data-protocol rule trivially true, avoids implementing hex symmetries differently per representation, and leaves augmentation as a clean additive ablation question.

Limits

Costs are single-machine; the wiring validation trained the graph arm at smoke scale only (its learning profile on gen-0 data matched the grid arm's, as expected for matched arms on shared data — not a comparison result); the controlled comparison itself is H6, under protocol v1.0.

Chapter 5 — Experimental protocol (booklet draft, 2026-09-26)

Source of authority: the frozen protocol v1.0 (D-020, sha256 f340a6b6...aefeb5) and the freeze/pin decisions D-017, D-019, D-025, D-026, D-027. This chapter restates them for the reader; the frozen document prevails on any divergence.

5.1 Pre-registration discipline

Everything that could bias the comparison was frozen before any comparison run existed, in this order: opponent population (D-017), protocol with its rejection rule and measured budgets (D-020), fixed shared openings (D-025); evaluation settings were pinned by config and test (D-019). The same-wall-clock cutoff T^* was computed by a rule stated in the pre-registered matrix — the median full-run wall-clock of the grid arm’s three runs — from grid clocks alone, before any cross-arm number was assembled. No frozen artifact was modified at any point; the repository history documents this.

5.2 Matrix, seeds, hardware, budgets

2 architectures $\times$ 3 training seeds (1–3; disjoint seed spaces), 10 generations $\times$ 500 self-play games per run at 128 full / 32 cheap simulations per decision (measured choices, not the plan’s placeholder values), 300-ply cap. One machine (Apple M1 Pro, 10 cores, 16 GB), four worker threads per run, runs sequential; each arm uses its measured best inference provider (grid: CoreML at 2.62 ms/eval; graph: CPU at 3.67 ms/eval) — a real hardware interaction reported and charged, not equalised away. Every run logs wall-clock per generation, states, and simulations; checkpoints are kept at every generation.

5.3 Opponents, openings, pairing

The evaluation population is frozen: seeded legal-random (B-RND), a documented fixed-weight heuristic (B-HEU; weights hash-pinned and test-enforced), and 6400-simulation no-network MCTS (B-MCTS). The gen-19 checkpoint of the prior demonstration loop is excluded (D-016/17). Openings are 250 pre-generated, unique, legal 4-ply openings (seeded generator, content hash recorded); every match plays pair i on opening line i with colours swapped, so all arms, seeds and opponents face identical opening/colour schedules (verified by schedule-hash test).

5.4 Metrics, intervals, exclusions

Primary metric: mean score (win 1 / draw 0.5 / loss 0) against the population, per (seed, opponent) cell, **excluding truncated games**, whose rate is always reported separately, with a truncations-as-0.5 sensitivity column and cap-treatment bounds (truncations as losses and as wins) bracketing any alternative cap. The resampled unit is the **seed**: cells aggregate games first; intervals are percentile bootstrap (10,000 resamples) over the three seeds; the arm contrast bootstraps both arms’ seed sets independently. Games are never pooled as i.i.d.; Elo, where printed by tooling, is descriptive only. Checkpoint selection is mechanical: generation-10 checkpoints for the same-examples reading; the last checkpoint completed at $\leq T^*$ for the same-wall-clock reading. **Exclusion criteria**: none were needed — no run failed, none was excluded, and a bad score is not an exclusion criterion by protocol.

5.5 RQ $\rightarrow$ experiment $\rightarrow$ result matrix

RQ	Experiment	Result artifact
RQ-H1: does the graph arm beat the grid arm at comparable budget?	The H6 matrix under both readings vs the frozen population	Per-seed score tables + arm contrast with seed-bootstrap intervals (results-same-examples / results-same-wallclock / results-arm-difference); verdict via the frozen rejection rule
RQ-H2 (secondary): per-example vs per-hour reading	The same campaign read at generation-10 vs T*-checkpoints	Score-vs-time curves (fig1); score/cost table (fig2)
RQ-H3 (secondary): what do graph components contribute?	Two one-component ablations at parity (A1 edge typing; A2 global pooling, substitution D-o28)	Ablation table H-T3 (results/ablations/); A1' supplementary (two-component, clearly labeled) if run

5.6 Reproduction

Every table and figure regenerates from scripts over raw per-game records (`scripts/make_results.py`, `make_figures.py`); every run's generator settings, seeds and model stamps live in per-run manifests; commands, configs and seeds are annexed. The minimal reproduction scenario (annex) replays a single recorded evaluation game deterministically and regenerates the result tables from the shipped records.

Results — The controlled comparison (report section draft)

Drafted 2026-09-19 at the H6 close (invariant 14). English working master; French copy at the report milestone (D-006). Every number traces to journals H6-2026-09-19-comparison-01 (3-seed analysis) and H6-2026-10-09-5seed-final-01 (final 5-seed analysis) and regenerates from `scripts/make_results.py / make_figures.py` over the raw per-game records; claims registered in `claims.md`.

The pre-registered question and its answer

The protocol (frozen at v1.0 before any comparison run) asked: at comparable training budget, does the graph architecture learn a better policy than the grid architecture for base-game Hive? It fixed, in advance, what would reject the hypothesis: no seed-consistent graph advantage under both budget readings, with intervals excluding a meaningful graph advantage.

That is what happened. H1 is rejected. Under the same-examples reading (both arms, 10 generations $\times$ 500 games) and the same-wall-clock reading ($T^* = 18.77$ h, computed by a pre-registered rule before any cross-arm number existed), the graph arm scored lower against two of the three frozen opponents and no better against the third, across **five seeds per arm** (seeds 4–5 were added after the three-seed analysis, symmetrically and under a pre-commitment to use all seeds regardless of direction, D-031; the three-seed analysis reached the same verdict):

graph – grid	Same-examples	Same-wall-clock
vs legal-random	–0.169 [–0.272, –0.062]	–0.161 [–0.278, –0.048]
vs heuristic	–0.064 [–0.111, –0.017]	–0.059 [–0.087, –0.029]
vs 6400-sim MCTS	–0.009 [–0.055, +0.028]	–0.018 [–0.066, +0.025]

(Seed-level means; bootstrap 95% over five seeds per arm; per-seed values in the results tables — no best-seed reporting anywhere. The largest upper bound across all six contrasts is +0.028.)

The wall-clock reading compounds the result: the graph arm’s measured cost was $2.0\times$ per run (36.7 vs 18.0 h of training wall-clock, mean over the five seeds), so at equal hours it completes only 3–6 of 10 generations (4–5 over the original three seeds) — a deficit the per-example reading already shows and equal time only widens.

Fig. 5 decomposes the aggregate scores into per-opponent trajectories for all 10 main-campaign runs; fig. 6 reports the training-fit metrics (policy and value) by generation — both arms fit their self-play data throughout, so the gap is not a bare optimization failure.

Truncation, reported separately and stress-tested

The clearest behavioural difference is not a score but an outcome category: against legal-random, the graph arm truncated 20–57% of its games at the 300-ply cap across the original seeds and 23–44% in the extension seeds (grid: 0–1%, with one extension seed at 11%) — winning material and then failing to convert (fig. 4, F1). Because truncation was defined as its own outcome from the start, this pathology is visible rather than laundered into draws; fig. 7 charts the per-run rates against legal-random. The cap value cannot rescue the hypothesis: even scoring every truncated game as a graph win — an upper

bound on any larger cap — leaves the graph arm behind on the random opponent under both readings (-0.072 / -0.058).

What this does and does not show

It shows: for a capacity-matched (+1.5%), simple relational message-passing network sharing every other component with the grid arm — rules, search, decoder, records, training conventions, frozen opponents, frozen openings, pinned evaluation — grid-plane encoding learned more per example AND per hour at this small budget, and the graph arm’s weakness concentrates in local tactical conversion, matching the local-vs-long-range asymmetry reported for Hex under DQN (Keller et al. 2023) now observed under matched AlphaZero-style self-play in a frameless, stacking game.

It does not show: anything about graph representations at larger budgets, other graph architectures, or other games; nor that the arms would not reorder with more generations (10 is early-regime; all scores against the strong baselines remain low). Five seeds per arm bound the statistics; the rejection is seed-consistent, and the extension seeds (added under pre-commitment) tightened four of the six intervals.

Costs (both denominations, per plan ch. 6)

Fig. 2: parameters 1.44M vs 1.47M; best-provider inference 2.62 ms (CoreML) vs 3.67 ms (CPU); self-play 13.0 vs 26.4 s/game (training wall-clock / 5,000 games); population score 0.411 vs 0.331 (same-examples), 0.405 vs 0.326 (same-wall-clock) — five-seed means.

Provenance

Population frozen 2026-09-09 (D-017); protocol frozen 2026-09-10 (D-020); openings frozen 2026-09-10 (D-025) — all before any comparison run. Campaign approved and launched 2026-09-10 (D-026); T* computed 2026-09-16 from grid wall-clocks only; no opponent, opening, or protocol touch occurred at any point. Negative result retained and reported per invariant 5 and protocol §1: the work does not need H1 confirmed to count.

The ablations (H-T3)

Two single-component ablations against the graph full method, three seeds each at full budget parity, plus one labeled supplement. Removing the direction-typed edge relations (A1, naive adjacency) destroyed trainability outright — NaN divergence in generation 0 for every seed — so typed edges carry, at minimum, the arm’s optimization stability. Removing the global-pooling bias (A2) produced a null: score changes of -0.001 [-0.170 , $+0.165$], -0.007 [-0.043 , $+0.030$] and -0.013 [-0.043 , $+0.013$] against the three opponents, with the failure modes unchanged. The supplementary A1’ (untyped edges plus gradient clipping — an explicitly two-component variant, never attributed to typing alone) trains stably and scores within the full arm’s band against all three opponents, suggesting the typed relations’ measurable contribution at this scale is concentrated in optimization stability rather than final strength — a statement that inherits the clip confound and is phrased accordingly wherever it appears.

Chapter 7 — Discussion and threats to validity (booklet draft, 2026-09-26)

Four-way split per plan ch. 21. Grounded in the claims register.

7.1 Reading the result

H1 was rejected exactly as the frozen protocol defined rejection. Three observations give the result its texture. First, the deficit is opponent-shaped: largest against legal-random, clear against the heuristic, absent against the search-heavy B-MCTS — consistent with a graph arm that is weakest at *local tactical conversion* rather than at positional judgment. Second, the truncation channel carries the mechanism: the graph arm wins material against random and then fails to close (20–57% capped games; fig4-F1), a pattern invisible in studies that fold caps into draws. Third, the ablations sharpen the reading: removing geometric edge typing does not merely weaken the graph arm — at parity settings it destroys trainability outright (NaN divergence, generation 0, 3/3 seeds), so the typed relations carry at minimum the optimization stability of the whole arm — while the second ablation found the global-pooling bias fully dispensable (score changes $\approx 0.00 \pm 0.17$, -0.01 ± 0.04 , -0.01 ± 0.03 against the three opponents; failure modes unchanged), localising the arm’s distinctive machinery in the directional relations, not the pooling. This contrasts instructively with the positive chess result of Rigaux & Kashima (2024, NeurIPS) and Keller et al.’s (2023) Hex asymmetry: our result does not contradict them — it bounds where their optimism transfers, and the comparison of evidence standards matters: the chess result rests on a single training run per model with intervals covering Elo estimation only, whereas the present rejection is seed-consistent across five runs per arm under two pre-registered budget readings. Hive’s tactics are dominated by short-range surround geometry, the regime Keller et al. found CNNs stronger in; at small budgets that regime decides games.

7.2 Internal validity (bugs, comparability)

The engine is validated independently of learning (perft to depth 6+ against published tables, 21/21 UHP conformance, 27,829-position differential agreement with two reference engines, a rules-derived hand-annotated corpus, 10.9M-transition invariant sessions). Both encoders are pinned byte-exactly against cross-language goldens run nightly. Arms share the decoder, records, losses, budgets and search; capacity differs by +1.5% (reported). Residual risks: the graph architecture is ONE point in design space — a stronger GNN might behave differently (we claim nothing beyond this net); tooling defects found during the study (an arena record-loss path on all-truncated matches; a silent NaN passage in ablation training) were caught by the verification discipline, fixed, and audited to have left campaign data untouched — but they illustrate that harness error, not chance, is the dominant failure mode at this scale.

7.3 Measurement validity (opponents, truncation)

The population is three fixed opponents spanning floor-to-mid strength; all trained arms still lose heavily to the two strong baselines, so the comparison lives in a low-score regime where differences against B-MCTS are hard to resolve. Scores are relative to THIS population — no universal strength claim is made. Truncation is handled as its own outcome with sensitivity bounds; the 300-ply cap itself cannot reverse the verdict (truncations-as-wins bound stays negative), but the high graph truncation rates mean its random-opponent score is measured on fewer decided games (43–80 per seed).

7.4 Statistical validity (seeds, dependencies)

Five seeds per arm bound the statistics — the pre-committed extension from three to five (D-031) tightened four of the six intervals and absorbed the most graph-favourable seeds observed without changing the verdict; the B-MCTS contrast remains indistinguishable from zero under both readings. The rejection does not rest on a single interval: it rests on seed-consistency across two opponents and two readings simultaneously, plus bounds analysis. Openings are shared across arms (pairing respected in the design); the arm contrast bootstraps seeds, not games; no game-level pseudo-replication enters any interval.

7.5 External validity (variant, hardware, budget)

One variant (base Hive), one machine, one small budget (10 generations $\times$ 500 games; $\sim$ 18–48 h/run), early-regime self-play throughout. The provider asymmetry (CoreML favours the convolutional net; the gather-heavy GNN runs fastest on CPU) is a genuine property of the deployment hardware, reported and charged — on different accelerators the wall-clock reading could shift. Nothing here generalises to other games, larger budgets, or richer graph architectures; the study answers its pre-registered question inside its pre-registered perimeter, and the negative answer is the product: encoding choice matters (as AZ-Hive found), and for Hive at small budget it favours the grid.

Chapter 8 — Working methodology: gated human–AI research (booklet, 2026-10-09)

Condensed from `docs/methodology.md` (workspace, v1.0) and the append-only methodology log; this chapter is also the report’s AI-use declaration in expanded form.

Division of labour

This study was executed under **goal-level delegation with mechanical human authority**. The human author is the principal investigator: he owns the research questions, every scientific commitment, every expenditure, everything public, and the final word on every claim — a responsibility that is not delegable. The AI assistant (Claude, Anthropic — operating as Claude Code sessions) is the runtime: given the research plan, it routes itself, builds, measures, journals, and drafts toward the plan’s end-state without per-task instruction.

The boundary is enforced by six **gates** enumerating decisions only the human makes: freezing any protocol, split or test set (G-FREEZE); spending or long compute (G-SPEND); anything leaving the machine (G-PUBLIC); reuse of material with unsettled rights (G-RIGHTS); institutional contact (G-ADMIN); destruction of data or results (G-DESTRUCTIVE). Every gate crossing in this study is recorded in the decision log with the human’s approval quoted verbatim — the protocol freeze, the opponent-population freeze with its budget sub-choice, the openings freeze, four compute approvals with explicit sizing, and the seed-extension pre-commitments.

Why the state lives in files

Sessions are stateless by design: a session reads the routing file, executes the active phase’s pipeline document (ordered tasks, each with an acceptance check, closed by exit criteria), journals everything measured, and updates the state files last. Research quality is thereby a property of **process artifacts** — journals, the decision log, frozen documents with hashes, the claims register — not of any session’s memory or competence. Everything in this booklet traces to those artifacts.

What the discipline caught

The methodology log records every incident where the discipline changed an outcome. During this study it caught, among others: a protocol- deadlock in the pipeline documents before any work ran; an engine-vs- corpus disagreement resolved *against* the hand-written corpus (the engine was right, and the record of being wrong was kept); an evaluation harness defect detected because three “different” opponents produced identical results; a silent record-loss path in the match runner found by the rule that every selected failure position must be *reproduced and verified* before publication; and an ablation that had silently diverged to NaN for ten generations, caught by the same identical-results alarm and converted into the study’s clearest ablation finding. The pattern is the methodology’s core claim: **at small scale, harness error is a larger threat than statistical noise, and only mechanical verification catches it.**

What the AI did not do

The AI chose no hypothesis, froze nothing, spent nothing, published nothing, and decided no claim. Where its drafts contained errors, the process — translation passes, mechanical consistency checks,

adversarial re-reads — surfaced several (stale seed counts, stale claim limits) before this version; the final-control record lists the checks. The human author has personally verified the conclusions he signs.

Chapter 9 — Conclusion (booklet draft, 2026-10-09)

No new results here, per the plan.

Within the tested perimeter — base-game Hive, one capacity-matched simple relational message-passing network against one grid CNN, ten generations of small-budget self-play, five seeds per arm, a frozen three-opponent population — **the answer to RQ-H1 is no**: the graph representation did not learn a better policy than the grid representation, under the same-examples reading or the same-wall-clock reading, and the pre-registered rejection rule fired exactly as frozen. The deficit is largest where Hive is most tactical, the graph arm pays twice the wall-clock, and its failure mode — winning material without converting — is visible precisely because truncation was never folded into draws. The result does not contradict the positive graph findings in Hex and chess; it bounds them: where short-range tactics decide games and budgets are small, frame artifacts are cheaper than framelessness.

Two follow-ups are motivated by the data rather than by hope. First, **the conversion pathology is a targetable defect**: the graph arm's value head learns respectably while its policy fails at forcing sequences, suggesting an experiment on search-time remedies (deeper evaluation budgets in won positions, or auxiliary targets for forcing moves) under the same frozen evaluation — a one-component change to the shared pipeline, applicable to both arms. Second, **the stability finding deserves isolation**: A1/A1' showed direction-typed relations matter mostly for optimization at this scale; a controlled study of normalisation and gradient-scale choices for relation-shared graph layers could decouple trainability from representational content — and would say whether naive adjacency, properly stabilised, is genuinely sufficient for Hive.

The broader method stands regardless of the sign of the result: pre-registration with frozen artifacts, dual budget readings, truncation as an outcome, seed-level inference, and write-as-you-go reporting turned a negative answer into a usable scientific object on a single laptop.

Bibliography and webographie (generated — plan ch. 27)

Generated by `scripts/make_bibliography.py` from the reading notes; every entry was actually consulted (its note is the evidence) and no entry appears in both sections.

Bibliography

- Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, Marc G. Bellemare. “Deep Reinforcement Learning at the Edge of the Statistical Precipice. NeurIPS 2021 (peer-reviewed; Outstanding Paper Award); arXiv:2108.13264. arXiv:2108.13264 (v1 Aug 30, 2021; v4 Jan 5, 2022), DOI 10.48550/arXiv.2108.13264; NeurIPS 2021 proceedings. Code: google-research/rliable (pip package rliable). (note: `docs/reading/agarwal-2021-precipice.md`, read 2026-09-09)
- Shai Ben-Assayag, Ran El-Yaniv. “Train on Small, Play the Large: Scaling Up Board Games with AlphaZero and GNN. arXiv preprint 2021 (arXiv:2107.08387 [cs.LG], v1 submitted 18 Jul 2021; no journal/conference version found as of 2026-09-26). arXiv:2107.08387v1 (18 Jul 2021), DOI 10.48550/arXiv.2107.08387; no code commit (code unreleased) (note: `docs/reading/benassayag-2021-scalable-alphazero.md`, read 2026-09-26)
- Tristan Cazenave, Yen-Chi Chen, Guan-Wei Chen, Shi-Yu Chen, Xian-Dong Chiu, Julien Dehos, Maria Elsa, Qucheng Gong, Hengyuan Hu, Vasil Khalidov, Cheng-Ling Li, Hsin-I Lin, Yu-Jin Lin, Xavier Martinet, Vegard Mella, Jeremy Rapin, Baptiste Roziere, Gabriel Synnaeve, Fabien Teytaud, Olivier Teytaud, Shi-Cheng Ye, Yi-Jun Ye, Shi-Jim Yen, Sergey Zagoruyko. “Polygames: Improved Zero Learning. ICGA Journal 42(4), pp. 244-256, 2020 (issue published Jan 2021); arXiv:2001.09832 (Jan 2020). DOI 10.3233/ICG-200157; arXiv:2001.09832; code repo archived 2022-03-02 (no pinned release cited) (note: `docs/reading/cazenave-2020-polygames.md`, read 2026-09-09)
- Danilo de Goede, Duncan Kampert, Ana Lucia Varbanescu. “The Cost of Reinforcement Learning for Game Engines: The AZ-Hive Case-study. ICPE 2022 (13th ACM/SPEC International Conference on Performance Engineering, Beijing), pp. 145-152. DOI 10.1145/3489525.3511685; open PDF at https://research.spec.org/icpe_proceedings/2022/proceedings/p145.pdf; BeeKeeper engine repo commit not pinned in paper (note: `docs/reading/degoede-2022-azhive.md`, read 2026-09-09)
- William L. Hamilton, Rex Ying, Jure Leskovec. Inductive Representation Learning on Large Graphs. NeurIPS 2017 (peer-reviewed; NIPS at the time). arXiv:1706.02216.. arXiv:1706.02216 (v4, 2018-09-10), DOI 10.48550/arXiv.1706.02216; NeurIPS 2017 proceedings. Code: github.com/williamleif/GraphSAGE (no specific release pinned). (note: `docs/reading/hamilton-2017-graphsage.md`, read 2026-09-09)
- Andy L. Jones. “Scaling Scaling Laws with Board Games. arXiv preprint 2021 (arXiv:2104.03113; v1 Apr 7, 2021, v2 Apr 15, 2021; not peer-reviewed). arXiv:2104.03113v2, DOI 10.48550/arXiv.2104.03113. Code/data: boardlaw project (andyljones.com/boardlaw, github.com/andyljones/boardlaw). (note: `docs/reading/jones-2021-scaling.md`, read 2026-09-09)
- Duncan Kampert, Ana-Lucia Varbanescu, Matthias Müller-Brockhausen, Aske Plaat. “Mimicking the Human Approach in the Game of Hive.” (preprint circulated as “Better AI for Hive: Mimicking human game-play strategies”) IEEE SSCI 2021 (IEEE Symposium Series on Computational Intelligence, Orlando, Dec 2021). IEEE Xplore document 9659999

(<https://ieeexplore.ieee.org/document/9659999/>); preprint PDF:
https://liacs.leidenuniv.nl/~plaata1/papers/IEEE_Conference_Hive_D_Kampert.pdf; dblp:
conf/ssci/KampertVMP21 (*note: docs/reading/kampert-2021-mimicking-hive.md, read 2026-09-09*)

- Yannik Keller, Jannis Blüml, Gopika Sudhakaran, Kristian Kersting. From Images to Connections: Can DQN with GNNs learn the Strategic Game of Hex? arXiv preprint 2023 (arXiv:2311.13414; submitted to ICLR via OpenReview id dYaeDrazj5, not listed as accepted — treat as non-peer-reviewed preprint).. arXiv:2311.13414 (2023-11-22), <https://arxiv.org/abs/2311.13414>; OpenReview forum dYaeDrazj5. Code: github.com/yannikkellerde/GNN_Hex (no release pinned). (*note: docs/reading/keller-2023-graphdqn-hex.md, read 2026-09-09*)
- Tomas Rigaux, Hisashi Kashima. “Enhancing Chess Reinforcement Learning with Graph Representation. NeurIPS 2024 (Advances in Neural Information Processing Systems 37, main conference track); arXiv:2410.23753. arXiv:2410.23753v1 (31 Oct 2024), DOI 10.48550/arXiv.2410.23753; NeurIPS proceedings DOI 10.52202/079017-0006; code repo not pinned to a commit in the paper (*note: docs/reading/rigaux-2024-chess-graph-rl.md, read 2026-09-26*)
- David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharmashan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, Demis Hassabis. “Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm.” Also published as: “A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play,” Science 362(6419):1140-1144, 2018. arXiv preprint 2017 (arXiv:1712.01815, v1 Dec 5, 2017); peer-reviewed version in Science 2018 (vol. 362, issue 6419, pp. 1140-1144). arXiv:1712.01815v1, DOI 10.48550/arXiv.1712.01815; Science version DOI 10.1126/science.aar6404. No official code release. (*note: docs/reading/silver-2017-alphazero.md, read 2026-09-09*)
- Elise van der Pol, Daniel E. Worrall, Herke van Hoof, Frans A. Oliehoek, Max Welling. MDP Homomorphic Networks: Group Symmetries in Reinforcement Learning. NeurIPS 2020 (peer-reviewed). arXiv:2006.16908.. arXiv:2006.16908 (v2, 2021-01-20), <https://arxiv.org/abs/2006.16908>; NeurIPS 2020 proceedings <https://papers.nips.cc/paper/2020/hash/2be5f9c2e3620eb73c2972d7552b6cb5-Abstract.html>. Code: github.com/ElisevanderPol/mdp-homomorphic-networks (no release pinned). (*note: docs/reading/vanderpol-2020-mdp-homomorphic.md, read 2026-09-09*)
- Oriol Vinyals, Meire Fortunato, Navdeep Jaitly. Pointer Networks. NeurIPS (NIPS) 2015, Advances in Neural Information Processing Systems 28 (peer-reviewed). arXiv:1506.03134.. arXiv:1506.03134 (v2, 2017-01-02), <https://arxiv.org/abs/1506.03134>; NIPS 2015 proceedings https://proceedings.neurips.cc/paper_files/paper/2015/hash/29921001f2fo4bd3baee84a12e98098-Abstract.html. (*note: docs/reading/vinyals-2015-pointer-networks.md, read 2026-09-09*)
- David J. Wu. “Accelerating Self-Play Learning in Go. arXiv preprint (arXiv:1902.10565; v1 Feb 27, 2019, v5 Nov 9, 2020); presented at the AAAI-20 Workshop on Reinforcement Learning in Games (not a full peer-reviewed proceedings paper). arXiv:1902.10565v5, DOI 10.48550/arXiv.1902.10565. Code: github.com/lightvector/KataGo (our reference reading: v5 paper; cite a specific release tag if we import any technique implementation). (*note: docs/reading/wu-2020-katago.md, read 2026-09-09*)

Webographie

- Jan Pfeifer (GitHub user janpfeifer). “hiveGo — Go Implementation of Hive Game.” GitHub repository. GitHub repository, 2018–2026 (created 2018-08-24, originally TensorFlow; refreshed 2025 onto GoMLX; last push 2026-08-20). main branch, commit d6ff95418d2b (2026-08-20,

“Updated main.wasm version.”); <https://github.com/janpfeifer/hiveGo>; consulted 2026-09-26
(note: *docs/reading/pfeifer-2025-hivego.md*, read 2026-09-26)

Annex A — Hand-annotated position corpora

Every expectation below was written by hand from the publisher's rules BEFORE any engine run (oracle-before-output); the engine run that followed is journaled, and the single disagreement found was resolved against the corpus (a One-Hive transit error in a setup sequence), with the engine vindicated. These renderings are generated from the executable case files by `scripts/make_annex_corpus.py` — the tests and the annex cannot drift apart.

A.1 Critical rules corpus (30 cases, H2)

Rules-correctness cases: placement, sliding/freedom to move, gates, stacking, One-Hive, terminal states, forced pass, and kernel-guard Pillbug stun cases.

Coo1 — Queen Bee may not be placed on the first turn (tournament rule)

- **Rule:** Tournament opening rule (*Tournament variant of the official rules (README source 4); Gen42 2010 rulesheet p. 3 alone would allow it*)
- **Setup:** ``
- **Expectation:** `move_illegal {'move': 'wQ'}`
- **Hand-written justification:** The 2010 base rulesheet says the Queen 'can be placed at any time from your first to your fourth turn' (p. 3), but the tournament variant — adopted by UHP and both reference engines, and the convention this study fixes — forbids placing the Queen Bee on either player's first turn. White's first move 'wQ' must therefore be rejected.

Coo2 — Queen must be placed on the fourth turn if not placed before

- **Rule:** Placing your Queen Bee (*Gen42 Hive rulesheet p. 3*)
- **Setup:** `wS1;bS1 wS1-;wG1 -wS1;bG1 bS1-;wA1 -wG1;bA1 bG1-`
- **Expectation:** `all_moves_place {'piece': 'wQ'}`
- **Hand-written justification:** 'You must place your Queen Bee on your fourth turn if you have not placed it before.' (p. 3). It is White's fourth turn and wQ is still in hand, so every legal move must be a placement of wQ. (Movement moves are additionally excluded by the Moving rule, p. 3: no moving before the queen is placed.) Setup legality: each white placement touches only white pieces, each black placement only black (Placing, p. 2).

Coo3 — No piece may move before that player's queen is placed

- **Rule:** Moving (*Gen42 Hive rulesheet p. 3*)
- **Setup:** `wS1;bS1 wS1-`
- **Expectation:** `all_moves_are_placements`
- **Hand-written justification:** 'Once your Queen Bee has been placed (but not before), you can decide whether to use each turn after that to place another tile or to move one of the pieces that have already been placed.' (p. 3). White's queen is unplaced on turn 2, so wS1 must have no movement moves — only placements are offered.

Coo4 — After the first pieces, placements may not touch the opponent's colour

- **Rule:** Placing (*Gen42 Hive rulesheet p. 2*)
- **Setup:** wG1;bS1 wG1/
- **Expectation:** moves_for_piece {'piece': 'wB1', 'moves': ['wB1 wG1\\', 'wB1 /wG1', 'wB1 -wG1']}
- **Hand-written justification:** ‘...with the exception of the first piece placed by each player, pieces may not be placed next to a piece of the opponent’s colour.’ (p. 2). wG1 sits at the origin with bS1 to its north-east. Of wG1’s five empty neighbours (E, SE, SW, W, NW), the E and NW cells each also touch bS1 (they are the two cells adjacent to both wG1 and its NE neighbour), so a new white piece may go only SE, SW or W of wG1. Expected wB1 placements: exactly those three cells. Hand geometry: axial E=(1,0), NE=(1,-1); neighbours of NE-cell (1,-1) include (1,0)=E-of-origin and (0,-1)=NW-of-origin.

Coo5 — Second player’s first piece joins the first piece (may touch enemy)

- **Rule:** Playing the Game / Placing (*Gen42 Hive rulesheet pp. 2-3*)
- **Setup:** wS1
- **Expectation:** moves_for_piece {'piece': 'bG1', 'moves': ['bG1 wS1-', 'bG1 wS1/', 'bG1 wS1\\', 'bG1 -wS1', 'bG1 /wS1', 'bG1 \\wS1']}
- **Hand-written justification:** ‘Play begins with one player placing a piece from their hand in the centre of the table and the next player joining one of their own pieces to it edge to edge.’ (p. 2) — the first-piece exception to the own-colour placing rule. Black’s first piece must join wS1 edge to edge, so bG1 may be placed on any of the six cells adjacent to wS1, and nowhere else.

Coo6 — Queen at the hive tip: exactly the two slides that keep contact

- **Rule:** Queen Bee / Freedom to Move / One Hive (contact) (*Gen42 Hive rulesheet pp. 4, 9, 10*)
- **Setup:** wS1;bS1 wS1-;wQ -wS1;bQ bS1-
- **Expectation:** moves_for_piece {'piece': 'wQ', 'moves': ['wQ \\wS1', 'wQ /wS1']}
- **Hand-written justification:** The Queen ‘can move only one space per turn’ (p. 4) in a sliding movement (p. 10), and ‘all pieces must always touch at least one other piece’ (p. 3 NB). wQ sits at the west tip of a straight line of four. Of its five empty neighbours, only the two cells that are also adjacent to its neighbour wS1 (the cells NW and SW of wS1) keep contact with the hive after the slide; the three cells further west touch nothing once the queen leaves. Neither destination is gated (each slide’s two flanking cells are one occupied, one empty). Expected: exactly those two moves.

Coo7 — Ant enclosed in a pocket: only exit is a gate, so it cannot move

- **Rule:** Freedom to Move (*Gen42 Hive rulesheet p. 10*)
- **Setup:** wA1;bS1 wA1-;wQ \\wA1;bQ bS1-;wG1 -wA1;bB1 bQ/;wS1 /wA1;bB1 bS1/;wB1 \\wQ;bB1 wA1/
- **Expectation:** moves_for_piece {'piece': 'wA1', 'moves': []}
- **Hand-written justification:** ‘If a piece is surrounded to the point that it can no longer physically slide out of its position, it may not be moved.’ (p. 10). Five of the ant’s six neighbours are occupied. The only empty neighbour (SE of the ant) is flanked by bS1 (E of the ant) and wS1 (SW of the ant) - the two cells adjacent to both the ant and that space - so the ant cannot physically slide into it. Removing the ant would NOT split the hive (the ring bB1-wQ-wG1-wS1 plus bS1 stays connected), so the block is purely freedom-to-move, not One Hive. The Ant, normally the most mobile piece, has zero legal moves.

Coo8 — Spider moves exactly three spaces along the hive edge - two destinations

- **Rule:** Spider (*Gen42 Hive rulesheet p. 7*)
- **Setup:** wG1;bS1 wG1-;wQ -wG1;bQ bS1-;wS1 \wQ;bG1 bQ-
- **Expectation:** moves_for_piece {'piece': 'wS1', 'moves': ['wS1 bS1/', 'wS1 wQ\\']} }
- **Hand-written justification:** 'The Spider moves three spaces per turn - no more, no less. It must move in a direct path and cannot backtrack on itself. It may only move around pieces that it is in direct contact with on each step.' (p. 7). The hive minus the spider is a straight five-piece line whose boundary is a single 14-cell ring with no gates; each ring cell touches the line, and cells off the ring touch nothing (excluded by the contact requirement). From its ring position the spider therefore has exactly two three-step walks - three cells clockwise and three cells anticlockwise: the cell NE of bS1, and the cell SE of wQ. One- and two-step stops are excluded ('no less'), backtracking is excluded.

Coo9 — Grasshopper: jumps only along occupied rows, no one-space slides

- **Rule:** Grasshopper (*Gen42 Hive rulesheet p. 6*)
- **Setup:** wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 \wS1;bG1 bQ-
- **Expectation:** moves_for_piece {'piece': 'wG1', 'moves': ['wG1 wS1\\', 'wG1 /wQ']} }
- **Hand-written justification:** 'It jumps from its space over any number of pieces (but at least one) to the next unoccupied space along a straight row of joined pieces.' (p. 6). The grasshopper touches occupied cells in exactly two of its six directions: SE (over wS1, landing in the next space, SE of wS1) and SW (over wQ, landing SW of wQ). In the other four directions the adjacent cell is empty, and a jump 'over at least one' piece is impossible - in particular the four adjacent empty cells are NOT destinations: the grasshopper 'does not move around the outside of the Hive like the other creatures'. Expected: exactly the two landing cells.

Coo10 — Grasshopper jumps a full five-piece row to the first empty space

- **Rule:** Grasshopper (*Gen42 Hive rulesheet p. 6*)
- **Setup:** wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 -wQ;bG1 bQ-
- **Expectation:** moves_for_piece {'piece': 'wG1', 'moves': ['wG1 bG1-']} }
- **Hand-written justification:** '...over any number of pieces (but at least one) to the next unoccupied space along a straight row of joined pieces.' (p. 6). Due east the grasshopper faces the unbroken row wQ, wS1, bS1, bQ, bG1; the first unoccupied space beyond it is the cell E of bG1 - the single destination. It must land there, not earlier (every nearer cell in the row is occupied). In all five other directions the adjacent cell is empty, so no jump exists. The grasshopper is a leaf of the hive, so One Hive does not restrict it.

Coo11 — Queen's only open neighbour is behind a gate: zero moves

- **Rule:** Freedom to Move (*Gen42 Hive rulesheet p. 10*)
- **Setup:** wS1;bS1 -wS1;wB1 wS1/;bQ -bS1;wQ wB1-;bG1 \bQ;wS2 wQ/;bA1 /bQ;wG1 -wS2;bS2 /bS1;wA1 wS2\;bB1 \bG1;wG2 wQ\;bG2 \bB1
- **Expectation:** moves_for_piece {'piece': 'wQ', 'moves': []} }
- **Hand-written justification:** 'Similarly, no piece may move into a space that it cannot physically slide into.' (p. 10). Five of the queen's six neighbours are white pieces; the sixth (the cell W of wG2, equally SE of wB1) is empty, but the two cells adjacent to both the queen and that space are wG2 and wB1 - both occupied - so the queen cannot physically slide in. Removing the queen

leaves the white horseshoe wS1-wB1-wG1-wS2-wA1-wG2 connected (and the black chain hangs off wS1 via bS1), so One Hive would allow the move; the block is purely Freedom to Move. Expected: the queen has no legal move.

Co12 — Ant reaches every cell of the hive perimeter (13 destinations)

- **Rule:** Soldier Ant / Freedom to Move (*Gen42 Hive rulesheet pp. 8, 10*)
- **Setup:** wG1;bS1 wG1-;wQ -wG1;bQ bS1-;wA1 \wQ;bG1 bQ-
- **Expectation:** moves_for_piece {'piece': 'wA1', 'moves': ['wA1 -wQ', 'wA1 \wG1', 'wA1 \bS1', 'wA1 \bQ', 'wA1 \bG1', 'wA1 bG1/', 'wA1 bG1-', 'wA1 bG1\\', 'wA1 bQ\\', 'wA1 bS1\\', 'wA1 wG1\\', 'wA1 wQ\\', 'wA1 /wQ']}
- **Hand-written justification:** ‘The Soldier Ant can move from its position to any other position around the Hive provided the restrictions are adhered to.’ (p. 8). The hive minus the ant is a straight five-piece line; its boundary is a single 14-cell ring with no gates (every slide step is flanked by one line cell and one empty cell), and every ring cell touches the line. The ant starts on the ring at the cell NW of wQ, so it can stop on any of the other 13 ring cells: the west cap (W of wQ), the five north-shoulder cells (NW of each line piece plus NE of bG1), the east cap (E of bG1), and the six south-shoulder cells (SE of each line piece plus SW of wQ). Cells off the ring touch no piece and are excluded (p. 3 NB: pieces must always touch at least one other piece).

Co13 — Beetle on the ground: two slides and two climbs

- **Rule:** Beetle (*Gen42 Hive rulesheet pp. 4-5, 10*)
- **Setup:** wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bG1 bQ-
- **Expectation:** moves_for_piece {'piece': 'wB1', 'moves': ['wB1 wS1', 'wB1 wQ', 'wB1 \bS1', 'wB1 \wQ']}
- **Hand-written justification:** ‘The Beetle, like the Queen Bee, moves only one space per turn. Unlike any other creature though, it can also move on top of the Hive.’ (p. 4). From (NW of wS1) the beetle may climb onto either adjacent piece - wS1 or wQ - or slide along the ground to the two empty cells that keep contact with the hive: NW of bS1 (touching wS1 and bS1) and NW of wQ (touching wQ). The two remaining empty neighbours touch no piece after the beetle lifts, so they are excluded (p. 3 NB). No gate blocks any of the four moves (each is flanked by at most one occupied cell, and for the climbs the flanking stacks are not taller than the destination). Exactly four moves - matching the rulesheet’s own beetle example count.

Co14 — Beetle on top of the hive: all six neighbouring cells

- **Rule:** Beetle (*Gen42 Hive rulesheet p. 5; beetle-gate ruling, World Hive Tournaments Rules FAQ*)
- **Setup:** wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bG1 bQ-;wB1 wS1;bA1 bG1-
- **Expectation:** moves_for_piece {'piece': 'wB1', 'moves': ['wB1 bS1', 'wB1 wQ', 'wB1 \wS1', 'wB1 \bS1', 'wB1 wS1\\', 'wB1 wQ\\']}
- **Hand-written justification:** ‘From its position on top of the Hive, the Beetle can move from tile to tile across the top of the Hive. It can also drop into spaces that are surrounded and therefore not accessible to most other creatures.’ (p. 5). Sitting on wS1, the beetle may move to every one of the six neighbouring cells: across onto bS1 or wQ (both height-1 stacks), or drop to any of the four empty cells around wS1 - each of which still touches wS1 itself, so contact holds. No pair of flanking stacks is taller than both origin (height 1 under the beetle) and destination, so no beetle gate applies (FAQ). One Hive cannot be violated: wS1 stays where it is. Exactly six destinations.

Co15 — Beetle gate: drop between two height-2 stacks is blocked

- **Rule:** Freedom to Move above ground level (beetle gate) (*World Hive Tournaments Rules FAQ; Gen42 Hive rulesheet p. 10*)
- **Setup:** wS1;bG1 wS1-/wQ /wS1;bQ bG1/wG1 wS1\;bB1 bQ-/wB1 -wS1;bB1 bQ/wB2 /wQ;bB1 bG1/wB1 wS1;bQ bB1-/wB2 wQ;bQ bB1/wB2 wG1;bA1 bQ/
- **Expectation:** move_illegal {'move': 'wB1 bB1\\'}
- **Hand-written justification:** 'When a piece climbs up or down the hive, or moves staying on top of the hive, it must be able to slide according to the freedom to move rule which applies to higher levels than the ground. If two stacks form a gate above the ground level (we call it beetle gate), pieces won't be able to slide through.' (WHT Rules FAQ). wB1 sits on wS1 (its own level: on top of a height-1 piece); the target cell SE of the bB1 stack is empty (height 0). The two cells adjacent to both origin and target carry the stacks bG1+bB1 and wG1+wB2, both height 2 - strictly taller than both the origin without the beetle (1) and the destination (0) - so the beetle cannot slide down between them. The drop must be rejected. (One Hive would allow it: wS1 stays in place; contact holds via the flanking stacks.)

Co16 — A piece with a beetle on top of it cannot move

- **Rule:** Beetle (stack immobility) (*Gen42 Hive rulesheet p. 5*)
- **Setup:** wS1;bS1 wS1-/wQ -wS1;bQ bS1-/wB1 \wS1;bG1 bQ-/wB1 wS1;bA1 bG1-
- **Expectation:** move_illegal {'move': 'wS1 bS1\\'}
- **Hand-written justification:** 'A piece with a beetle on top of it is unable to move' (p. 5). wS1 lies under wB1, so any attempt to move wS1 - here a spider move towards the cell SE of bS1 - must be rejected, regardless of whether the path would otherwise be legal for a spider.

Co17 — Stack takes the beetle's colour: white may place beside a covered black queen

- **Rule:** Beetle (stack colour) / Placing (*Gen42 Hive rulesheet pp. 2, 5*)
- **Setup:** wS1;bS1 wS1-/wQ -wS1;bQ bS1-/wB1 \wS1;bA1 bS1\;wB1 \bS1;bG1 bA1\;wB1 \bQ;bG2 bG1\;wB1 bQ;bB1 bG2\
- **Expectation:** move_legal {'move': 'wG1 wB1-'}
- **Hand-written justification:** '...for the purposes of the placing rules on p. 2, the stack takes on the colour of the Beetle.' (p. 5). White's beetle sits on the black queen at the east end of the hive. The cell E of that stack touches no other piece, so a white placement there is adjacent only to a stack whose colour is - by the rule - white. Placing wG1 there must be accepted. (Without the stack-colour rule the cell would be adjacent to a black piece and the placement would be illegal, p. 2.)

Co18 — Stack takes the beetle's colour: black may NOT place beside its own covered queen

- **Rule:** Beetle (stack colour) / Placing (*Gen42 Hive rulesheet pp. 2, 5*)
- **Setup:** wS1;bS1 wS1-/wQ -wS1;bQ bS1-/wB1 \wS1;bA1 bS1\;wB1 \bS1;bG1 bA1\;wB1 \bQ;bG2 bG1\;wB1 bQ;bB1 bG2\;wG1 -wQ
- **Expectation:** move_illegal {'move': 'bB2 wB1-'}
- **Hand-written justification:** Mirror of Co17: the stack bQ+wB1 counts as WHITE ('the stack takes on the colour of the Beetle', p. 5). The cell E of the stack touches only that stack, so for black it is adjacent to a white piece and 'pieces may not be placed next to a piece of the opponent's

colour' (p. 2). Black's attempt to place bB2 there must be rejected - even though the buried piece is black's own queen.

Co19 — One Hive: the only connection between two parts may not move

- **Rule:** One Hive rule (*Gen42 Hive rulesheet pp. 3, 9*)
- **Setup:** wS1;bS1 wS1-;wQ -wS1;bQ bS1-
- **Expectation:** moves_for_piece {'piece': 'wS1', 'moves': []}
- **Hand-written justification:** 'All pieces must always touch at least one other piece. If a piece is the only connection between two parts of the Hive, it may not be moved.' (p. 3 NB); 'The pieces in play must be linked at all times. At no time can you leave a piece stranded (not joined to the Hive) or separate the Hive in two.' (p. 9). wS1 is the interior link between wQ on one side and bS1-bQ on the other: lifting it splits the hive, so the spider has no legal move at all - every destination, however valid as spider movement, is excluded by One Hive.

Co20 — Ring: a piece on a closed loop may move (not a cut point); the ring's eye is gated

- **Rule:** One Hive rule / Freedom to Move (*Gen42 Hive rulesheet pp. 9, 10*)
- **Setup:** wS1;bS1 -wS1;wG1 wS1/;bQ -bS1;wQ wS1\;bG1 -bQ;wG2 wG1-;bG2 -bG1;wA1 wQ-;bA1 -bG2;wS2 wG2\;bB1 -bA1
- **Expectation:** moves_for_piece {'piece': 'wQ', 'moves': ['wQ /wS1', 'wQ /wA1']}
- **Hand-written justification:** The six white pieces form a closed ring, so removing wQ leaves the other five connected around the loop (and the black tail hangs off wS1): One Hive permits the queen to move. Sliding one space (p. 4), the queen has three empty neighbours: the ring's eye and two outside cells. The eye is flanked by wS1 and wA1 - both occupied - so the queen 'may not move into a space that it cannot physically slide into' (p. 10). The two outside cells (SW of wS1, which touches wS1 and bS1; and SW of wA1, which touches wA1) are unobstructed slides keeping contact. Expected: exactly those two destinations.

Co21 — Game ends when a queen is fully surrounded - even by its own colour

- **Rule:** The Object of Hive / The End of the Game (*Gen42 Hive rulesheet pp. 1, 11*)
- **Setup:** wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 -wQ;bG1 bQ-;wA1 -wG1;bG2 bQ/;wA2 -wA1;bB1 \bQ;wA3 -wA2;bA1 bS1\;wS2 -wA3;bA2 bQ\
- **Expectation:** game_over {'state': 'WhiteWins'}
- **Hand-written justification:** 'The pieces surrounding the Queen Bee can be made up of a mixture of both your pieces and your opponent's.' (p. 1). 'The game ends as soon as one Queen Bee is completely surrounded by pieces of any colour. The person whose Queen Bee is surrounded loses the game.' (p. 11). Black's final placement (bA2, SE of its own queen) fills the sixth and last cell around bQ. The surrounding pieces are all black - irrelevant per p. 1 - and it is Black's own move that completes the surround: Black loses, the GameString state must read WhiteWins immediately after that move.

Co22 — One move surrounds both queens simultaneously: draw

- **Rule:** The End of the Game (*Gen42 Hive rulesheet p. 11*)
- **Setup:** wS1;bS1 wS1/;wQ wS1\;bB1 bS1-;wA1 -wQ;bQ bB1\;wS2 /wQ;bQ /bB1;wG1 -wS2;bQ wQ-;wB1 -wA1;bA1 bQ\;wG1 wS2-;bG1 \bS1;wB1 \wA1;bA2 bQ-;wB1 \wS1;bA3 bB1\;wG2 -

wB1;bG1 bS1\

- **Expectation:** game_over {'state': 'Draw'}
- **Hand-written justification:** 'The person whose Queen Bee is surrounded loses the game, unless the last piece to surround their Queen Bee also completes the surrounding of the other Queen Bee. In that case the game is drawn.' (p. 11). Before Black's last move, each queen has exactly one empty neighbour - the same cell (1,0), adjacent to both queens (the queens sit side by side, wQ NE-shoulder wS1, bQ beside it). Black's grasshopper at (1,-2) jumps SE over bS1 into that cell, filling the sixth neighbour of both queens with a single piece: the game must end as a Draw, not a win for either side.

Co23 — A player who can neither place nor move must pass

- **Rule:** Unable to move or place (*Gen42 Hive rulesheet pp. 2, 5, 10, 11*)
- **Setup:** wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bQ bS1/;wA1 /wQ;bQ bS1-;wB1 \bS1;bQ bS1/;wB1 bS1;bQ wB1-;wA1 /bQ;bQ wB1/;wA2 /wQ;bQ wB1-;wA2 bQ\;bQ wB1/;wA3 /wQ;bQ wB1-;wA3 bQ-;bQ wB1/;wG1 /wB1;bQ wB1-;wG1 wB1/
- **Expectation:** must_pass
- **Hand-written justification:** 'If a player can neither place a new piece or move an existing piece, the turn passes to their opponent who then takes their turn again.' (p. 11). After White's final move Black has: bS1 under wB1 - 'a piece with a beetle on top of it is unable to move' (p. 5), and the stack counts as white for placing (p. 5); and bQ, whose five neighbours are the stack, wG1, wA1, wA2 and wA3, with its single empty neighbour reachable only between wG1 and wA3 - a gate the queen 'cannot physically slide into' (p. 10). No placement is possible either: the only cell adjacent to a black-topped piece is that same gated cell, which also touches white pieces (p. 2). Black is not lost - bQ has an empty neighbour, so it is not surrounded - but must pass.

Co24 — Pillbug special ability: moving an adjacent friendly piece (kernel guard)

- **Rule:** Pillbug special ability (*Gen42 Pillbug rulesheet (English section)*)
- **Setup:** wP;bS1 wP-;wQ -wP;bQ bS1-
- **Expectation:** move_legal {'move': 'wQ wP\\'}
- **Hand-written justification:** 'The special ability allows the Pillbug to move an adjacent piece (friend or enemy) two spaces; up onto itself and then down into another empty space adjacent to itself.' (Pillbug rulesheet). wQ is adjacent to wP; the target cell SE of wP is empty and adjacent to wP. None of the four exceptions applies: wQ was not just moved by the other player (Black's last move was placing bQ), wQ is not in a stack, removing wQ does not split the hive (it is a leaf), and no stacked pieces form a gap on the up-and-over path. NOTE: this is a variant-tagged kernel-guard case (protocol §2) - the study variant is base game; Pillbug cases only protect the shared rules kernel.

Co25 — A piece just moved by the enemy pillbug is stunned for one turn (kernel guard)

- **Rule:** Pillbug special ability (immobility of the moved piece) (*Gen42 Pillbug rulesheet (English section); World Hive Tournaments Rules FAQ*)
- **Setup:** wS1;bP wS1-;wQ -wS1;bQ bP-;wA1 \wQ;bG1 bQ-;wA1 \bP;bG1 -wQ;wG1 -wA1;wA1 bP\
- **Expectation:** move_illegal {'move': 'wA1 bP/'}

- **Hand-written justification:** ‘Furthermore, any piece moved by the Pillbug may not be moved at all (directly or via Pillbug action) on the next player’s turn.’ (Pillbug rulesheet); FAQ: ‘any piece that just moved, in the turn of the other player immediately after is unable to: move, be moved or use the pillbug’s ability.’ Black’s pillbug just threw wA1 up over itself and down to the cell SE of bP (a legal use: wA1 last moved two plies earlier, so the just-moved exception did not block the throw; removing it kept the hive whole since wG1 also touches wS1 and wQ). On White’s very next turn the thrown ant is stunned: the attempted ant move to NE of bP must be rejected. Variant-tagged kernel-guard case (protocol §2).

Co26 — Pillbug may not move the piece the opponent just moved (kernel guard)

- **Rule:** Pillbug special ability (exceptions) (*Gen42 Pillbug rulesheet (English section)*)
- **Setup:** wS1;bP wS1-;wQ -wS1;bQ bP-;wA1 \wQ;bG1 bQ-;wA1 \bP
- **Expectation:** move_illegal {'move': 'wA1 bP\\'}
- **Hand-written justification:** ‘The Pillbug may not move the piece which was just moved by the other player.’ (Pillbug rulesheet, first exception). White’s ant moved to the cell NW of bP on the immediately preceding ply; Black’s attempt to use the pillbug’s ability on that same ant - throwing it to SE of bP - must be rejected. (The identical throw becomes legal two plies later, which is case Co25’s setup.) Variant-tagged kernel-guard case (protocol §2).

Co27 — One Hive binds even the grasshopper: a cut-point cannot jump

- **Rule:** One Hive rule / Grasshopper (*Gen42 Hive rulesheet pp. 3, 6, 9*)
- **Setup:** wG1;bS1 wG1-;wQ -wG1;bQ bS1-
- **Expectation:** moves_for_piece {'piece': 'wG1', 'moves': []}
- **Hand-written justification:** The grasshopper is exempt from the sliding restriction (p. 10: it ‘can jump into or out of a space’), but not from One Hive: ‘If a piece is the only connection between two parts of the Hive, it may not be moved.’ (p. 3 NB). wG1 sits between wQ and the black pair; lifting it for any jump splits the hive in two, so despite having jump lines in both E and W directions the grasshopper has no legal move.

Co28 — A beetle cannot be PLACED directly on top of the hive

- **Rule:** Beetle (placement NB) (*Gen42 Hive rulesheet p. 5*)
- **Setup:** wS1;bS1 wS1-;wQ -wS1;bQ bS1-
- **Expectation:** move_illegal {'move': 'wB1 wS1'}
- **Hand-written justification:** ‘When it is first placed, the Beetle is placed in the same way as all the other pieces. It cannot be placed directly on top of the Hive, even though it can be moved there later.’ (p. 5 NB). wB1 is still in hand; the attempt to introduce it on top of wS1 must be rejected. (Co13/Co14 verify that the same beetle may climb there by a move once placed.)

Co29 — Spider may not stop after one step (‘no more, no less’)

- **Rule:** Spider (*Gen42 Hive rulesheet p. 7*)
- **Setup:** wG1;bS1 wG1-;wQ -wG1;bQ bS1-;wS1 \wQ;bG1 bQ-
- **Expectation:** move_illegal {'move': 'wS1 \\wG1'}
- **Hand-written justification:** ‘The Spider moves three spaces per turn - no more, no less.’ (p. 7). The cell NW of wG1 is exactly one sliding step from the spider’s position, and no legal three-step non-backtracking path ends there (the two three-step walks end NE of bS1 and SE of wQ - case

Coo8); a path through that cell passes it at step one and may not stop. The one-step move must be rejected.

Coo8 — Queen between two pieces: two slides along the shoulder

- **Rule:** Queen Bee / One Hive / Freedom to Move (*Gen42 Hive rulesheet pp. 4, 9, 10*)
- **Setup:** wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bG1 bQ-
- **Expectation:** moves_for_piece {'piece': 'wQ', 'moves': ['wQ -wB1', 'wQ /wS1']}
- **Hand-written justification:** The queen touches wS1 (E) and wB1 (NE). Removing her keeps the hive whole (wB1 still touches wS1), so One Hive allows a move. One-space slides (p. 4): of her four empty neighbours, only the cell W of wB1 (keeping contact with wB1) and the cell SW of wS1 (keeping contact with wS1) still touch the hive after she lifts; the two far-western cells touch nothing and are excluded (p. 3 NB). Neither slide is gated (each flanked by exactly one occupied cell). Expected: exactly those two destinations.

A.2 Tactical verification set (5 cases, H3)

Search-correctness cases: mate-in-1 by walk and by jump from both colours, and self-surround avoidance; solved 5/5 by the MCTS baseline at 400, 1600 and 6400 simulations.

Too1 — White mates in 1: occupy the black queen's last liberty (SE of bQ)

- **Rule:** The End of the Game (*Gen42 Hive rulesheet pp. 1, 11*)
- **Setup:** wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 -wQ;bG1 bQ-;wA1 -wG1;bG2 bQ/;wA2 -wA1;bB1 \bQ;wA3 -wA2;bA1 bS1\
- **Expectation:** bestmove_to_cell {'target': 'bQ\\'}
- **Hand-written justification:** The black queen has exactly one empty neighbour, the cell SE of bQ. Any white piece landing there completes the surround and wins immediately ('the game ends as soon as one Queen Bee is completely surrounded by pieces of any colour', p. 11; mixture of colours allowed, p. 1). The cell is reachable: a white ant can walk the south perimeter in one move (entry past bA1 is not gated), so a winning move exists. No other single move ends the game. The searcher must play onto that cell.

Too2 — Black mates in 1: occupy the white queen's last liberty (SE of wQ)

- **Rule:** The End of the Game (*Gen42 Hive rulesheet pp. 1, 11*)
- **Setup:** wS1;bS1 -wS1;wQ wS1-;bQ -bS1;wG1 wQ-;bG1 -bQ;wG2 wQ/;bG2 -bG1;wB1 \wQ;bA1 -bG2;wA1 wS1\;bA2 -bA1;wA2 wG1-
- **Expectation:** bestmove_to_cell {'target': 'wQ\\'}
- **Hand-written justification:** Mirror of Too1 with the colours exchanged and black to move — this pair is the player-alternation check at the move level: the winning pattern must be found from both sides. The white queen's only empty neighbour is the cell SE of wQ; a black ant reaches it along the south perimeter (route via SE of bS1's column: the step into the cell is flanked by the occupied wA1 cell, so contact holds and no gate blocks). Landing there completes the surround: Black wins (p. 11).

Too3 — White mates in 1 by grasshopper jump over four pieces (E of bQ)

- **Rule:** Grasshopper / The End of the Game (*Gen42 Hive rulesheet pp. 6, 11*)
- **Setup:** wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 -wQ;bG1 bQ/;wA1 \wQ;bB1 \bQ;wA2 \wA1;bA1 bS1\;wA3 \wA2;bA2 bQ\

- **Expectation:** `bestmove_to_cell {'target': 'bQ-'}`
- **Hand-written justification:** The black queen's only empty neighbour is the cell E of bQ. Due east from wG1 at the west cap runs the unbroken occupied row wQ, wS1, bS1, bQ; the next unoccupied space along that row is exactly the winning cell, so the grasshopper jumps over four pieces and completes the surround (p. 6: 'over any number of pieces ... to the next unoccupied space along a straight row of joined pieces'; p. 11: surround ends the game). White ants can also walk in around the perimeter — the expectation is the destination cell, whichever piece the searcher sends.

To04 — Black mates in 1 by grasshopper jump over four pieces (W of wQ)

- **Rule:** Grasshopper / The End of the Game (*Gen42 Hive rulesheet pp. 6, 11*)
- **Setup:** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 \wQ;bG1 bQ-;wG2 \wS1;bA1 bQ\;wA1 wQ\;bA2 bA1\;wA2 /wQ;bA3 bA2\;wB1 \wG1`
- **Expectation:** `bestmove_to_cell {'target': '-wQ'}`
- **Hand-written justification:** Alternation mirror of To03 with black jumping. The white queen's neighbours: E wS1, NW wG1 (placed NW of wQ), NE wG2 (placed NW of wS1 = NE of wQ), SE wA1, SW wA2 — five occupied, only W of wQ empty. Due east of that gap runs the unbroken row wQ, wS1, bS1, bQ with bG1 at the east cap (3,0): from bG1 the next unoccupied space westward along the row is exactly the gap, so the grasshopper jumps over four pieces and completes the surround (pp. 6, 11). Black's southern tail (bA1..bA3 SE of bG1) keeps black's earlier placements legal and away from white.

To05 — Do not fill your own queen's last liberty

- **Rule:** The End of the Game (*Gen42 Hive rulesheet p. 11*)
- **Setup:** `wS1;bS1 -wS1;wQ wS1-;bQ -bS1;wG1 wQ-;bG1 -bQ;wG2 wQ/;bG2 -bG1;wB1 \wQ;bA1 -bG2;wA1 wS1\;bA2 -bA1`
- **Expectation:** `bestmove_avoid_cell {'target': 'wQ\\'}`
- **Hand-written justification:** The cell SE of wQ is the white queen's last liberty. White placing or moving any piece there completes the surround of White's own queen — 'the person whose Queen Bee is surrounded loses the game' (p. 11) regardless of who supplied the sixth piece. The placement is perfectly legal (the cell touches only white pieces), so only search judgment prevents it. Any move except one landing on that cell passes; the searcher must not play into it. (This does not assert White survives long-term — black threatens the same cell — only that the immediate self-kill is avoided.)

Annex B — Architectures, decoder and data formats in detail

Sources: `python/hivenet/model.py`, `python/hivenet/graph_model.py`, `crates/hive-nn/` (authoritative); `docs/representations/`; `docs/action-decoder.md`. Parameter counts are measured (`count_params`), not estimated.

B.1 Grid arm — HiveNet (1.44 M parameters)

Input: 77 planes $\times$ 32 $\times$ 32 (float32 in [0,1]). Plane groups: 64 piece planes (own/opponent $\times$ 8 bug types $\times$ stack level 0–3+), one-hive-pinned tops, last-moved (stun) cell, legal placement regions for both sides, side-to-move constant, queen-liberty scalars (both queens, /6), ply/100, game-type bits, reserve counts (/14).

Body: 3 $\times$ 3 convolutional stem $\rightarrow$ 8 residual blocks of 96 channels (two 3 $\times$ 3 conv + BatchNorm each); blocks 2 and 5 carry a KataGo-style global-pooling bias (mean|max pooled channels $\rightarrow$ linear $\rightarrow$ per-channel bias). Hex adjacency on axial coordinates is a 7-cell subset of the 3 $\times$ 3 neighbourhood, so plain 3 $\times$ 3 convolutions cover it; the two non-neighbour corners become learnable dead weights.

Heads: policy = 1 $\times$ 1 convolution to 28 piece-slot planes, flattened to 28,672 spatial logits, plus a pass logit from pooled features (POLICY_SIZE = 28,673); value = pooled features $\rightarrow$ 64 $\rightarrow$ 3 (win/draw/loss from the side to move).

B.2 Graph arm — HiveGraphNet (1.47 M parameters, +1.5%)

Input per position: up to 224 nodes (occupied cells + every empty cell adjacent to the hive — exactly the decoder’s destination universe), each with 56 features: per stack level 0–4 a (present, owner-is-mover, bug one-hot[8]) block; stack height/5; empty-candidate, one-hive-pinned, last-moved, and both placement-region bits. A 23-value global vector (side, ply/100, queen liberties/6, per-bug-type reserves/3 for both sides, game-type bits) is concatenated to every node’s input.

Body: input linear to 152 channels $\rightarrow$ 8 relational message-passing layers: $h'_i = \text{ReLU}(W_{\text{self}} h_i + \sum_d W_d h_{\{n_i(d)\}} + b)$ with six direction-typed weight matrices (the six hex directions as edge types), residual, masked; every third layer adds a masked global-pooling bias (mean|max $\rightarrow$ linear). Neighbour structure is a (224 $\times$ 6) index tensor with a zero pad row — all shapes static, so the ONNX export is fixed-shape and runs under the same Rust inference path as the grid arm.

Heads: value = masked mean|max pooling | globals $\rightarrow$ 64 $\rightarrow$ 3 (same convention). Policy = per-candidate scoring through the shared decoder: for each legal (slot, destination) the logit is MLP(dest-node embedding | source embedding | slot embedding[32]), where the source embedding is the piece’s standing node for movements and a learned reserve vector for placements; one learned pass logit. Logits attach to moves, never to list positions.

Ablation variants (H-T3): `untyped_edges` shares ONE matrix across the six directions (0.54 M — the typed matrices are the ablated component); `no_gpool` removes every pooling bias (1.37 M).

B.3 The shared action decoder

A move is (side-relative piece slot 0–27, destination cell) plus pass; slots 14–27 address opponent pieces (pillbug throws; inert in base game). The grid arm materialises the space as the flat 28,673-way tensor (slot $\times$ frame cell); the graph arm scores the identical pairs per candidate. Both arms: identical legal-set masking, softmax over exactly the legal set, identical MCTS visit-distribution targets, deterministic tiebreak toward the lowest flat index. Check 1 of the pre-training battery asserts zero probability mass outside the legal set through a real forward pass for whichever arm is under test.

B.4 Record format v3 (818 bytes)

Bytes 0–83: $28 \times (x, y, \text{level})$ in frame coordinates, 255 = in hand; 84 side to move; 85 last-moved id (stun state); 86 ply; 87 game-type bits; 88–91 queen liberties and reserves (mover/opponent); 92–95 one-hive-pinned bitmask; 96–97 played-move policy index; 98 outcome from the mover’s perspective — 0 loss / 1 draw / 2 win / **3 truncated** (never a draw); 99 version; 100–107 generating-model stamp (generation, net hash); 112–175 top-15 MCTS visit distribution + total; 176–177 legal-move count; 178–817 the legal policy-index list (cap 320; measured max branching 213) that enables legal-masked training in both arms. Rust and Python builders for both the plane and graph encodings are pinned byte-identical by nightly cross-language golden tests (240 and 160 positions respectively).

B.5 Search (shared)

PUCT MCTS, $c = 1.4$, batched leaf evaluation, terminal values backed up exactly; self-play adds Dirichlet root noise (ϵ 0.25), temperature sampling for 12 plies, playout-cap randomization (25% of decisions at 128 simulations — recorded; 75% at 32 — unrecorded), resignation at -0.92 with a 10% no-resign audit; evaluation runs 400 simulations, no noise, deterministic argmax (enforced by test, pinned by config).

Annex C — Hyperparameters, seeds, commands: full reproduction sheet

Every value below is the one the campaigns actually ran (per-run `train-config.json` and `*-manifest.json` files are authoritative); nothing here is a recommendation.

C.1 Frozen experimental constants

Constant	Value	Frozen by
Variant	base game, tournament opening rule	D-007 / protocol v1.0
Move cap	300 plies; truncation = own outcome	D-008/D-011 / protocol §3
Self-play sims (full/cheap)	128 / 32, full-frac 0.25	protocol §5
Temperature plies / resign / audit	12 / -0.92 / 10%	matrix (pre-registered)
Eval sims / noise / volume	400 / none / 100 games/opponent	D-019, D-027
Opponent population	B-RND, B-HEU (weights sha256 d0602f18...0b97a), B-MCTS@6400	D-017
Openings	250 × 4-ply, content sha256 63b318do...5af7b	D-025
T* (same-wall-clock cutoff)	18.77 h (median of grid s1–s3 totals)	pre-registered rule, computed 2026-09-16
Seeds	1–5 per arm (4–5 added under pre-commitment)	D-026 / D-031

C.2 Training hyperparameters (identical both arms)

SGD, lr 0.02 cosine-annealed to lr/100 over the run, momentum 0.9, weight decay 1e-4, batch 256, 2 epochs per generation, value-loss weight 0.6, legal-masked policy cross-entropy against MCTS visit distributions, truncated records excluded from the value loss. Grid: channels 96 × blocks 8. Graph: hidden 152 × layers 8, slot embedding 32. No hyperparameter search was performed for either arm (same zero tuning budget, plan ch. 6); the only post-hoc optimizer change in the whole study is the gradient clip of the explicitly two-component A1' supplement.

C.3 Seed derivation

Training run (arm, seed): `base_seed = 100,000 × seed`; generation `g` uses `base_seed + g` for self-play and training; the gen-o network is a seeded random initialisation exported to ONNX (`torch.manual_seed = base_seed`). Evaluation: network seed 9000+gen (finals) / 9500 (T* sets), opponent seeds 9101 (B-RND) and 9201 (B-MCTS), arena seeds as logged; with fixed openings the schedule is seed-independent (proven by schedule-hash test).

C.4 Commands

```

# one training run of the matrix (resumable per generation)
python3 scripts/run_comparison.py --arm grid --seed 1 --gens 10 --games 500
python3 scripts/run_comparison.py --arm graph --seed 1 --gens 10 --games 500
# ablations: --arm graph-untyped | graph-nogpool | graph-untyped-clip

# regenerate every table and figure from raw per-game records
python3 scripts/make_results.py
python/.venv/bin/python scripts/make_figures.py

# pre-training check battery on any shard set
bash scripts/run_h4_checks.sh '<abs>/gen000-*.bin' '<abs>/gen000-manifest.json'

# fresh-environment minimal reproduction (clone, build, replay, compare)
bash scripts/reproduce_minimal.sh /tmp/repro

# booklet (assembly -> HTML -> PDF)
python3 scripts/assemble_booklet.py && bash scripts/make_booklet_pdf.sh

# French/English numeric-identity check
python3 scripts/check_fr_numbers.py

```

C.5 Measured machine profile (all wall-clock figures)

Apple M1 Pro (10 cores, 16 GB, macOS 15.3.1); 4 worker threads per run, runs sequential under `caffeinate`. Inference: grid CoreML 2.62 ms/eval, graph CPU 3.67 ms/eval (CoreML slower for the gather-heavy graph net — measured, reported, charged). Training throughput benchmark on MPS (batch 128, forward+backward): 274 (grid) / 138 (graph) pos/s. Training wall-clock per 10×500 -game run (self-play generation + training, evaluation excluded): grid 16.7–19.1 h, graph 26.6–49.9 h; evaluation ≈ 23 –32 s/game at 400 sims.

C.6 Journal and decision index for this study

Protocol and freezes: D-007/008/011/012/017/019/020/025. Campaigns: D-026 (main), D-029 (ablations), D-030 (A1'), D-031 (5-seed extension with pre-commitments). Analysis journals: H6-2026-09-19-comparison-01 (3-seed), H6-2026-10-09-5seed-final-01 (final), H7-2026-09-23 / 10-02 / 10-09 (A1, A2, A1'). Engine validation: H2-2026-09-09 set. Every table cell in this booklet is reachable from one of these.

Annex D — Repository pointers

Everything not materialised in annexes A–C lives in the repository; each pointer names its identifiers and how to read them (plan ch. 23).

- **D.1 Configs, seeds, manifests.** `configs/` (matrix, baselines with sha256-pinned weights, pinned eval settings, ablation diffs); per-run `*-manifest.json` and `wallclock.json` under `data/runs/`.
- **D.2 Encoding conventions in full.** `docs/representations/{grid,graph, comparison-controls}.md`; `docs/action-decoder.md` — the normative prose behind annex B.
- **D.3 Claims register.** `paper/claims.md` — one row per claim: claim, evidence, section, limit. No row, no claim.
- **D.4 Experiment journal.** `journal/` — every run and measurement, negative results included; decision log in `state/decisions.md` (workspace repository).

Annex E — Result tables and figures (generated)

Pulled verbatim at assembly time from `results/` and `paper/figures/` — regenerate via `scripts/make_results.py` and `scripts/make_figures.py`.

H6 results — same-examples reading

Score = mean over non-truncated games (win 1 / draw 0.5 / loss 0); trunc = truncation rate; sens = truncations scored 0.5.

Arm	Seed	B-RND score / trunc	B-HEU score / trunc	B-MCTS score / trunc
grid	s1	0.995 / 0%	0.150 / 0%	0.125 / 0%
grid	s2	0.975 / 1%	0.080 / 0%	0.125 / 0%
grid	s3	0.990 / 1%	0.190 / 0%	0.075 / 0%
grid	s4	0.949 / 11%	0.105 / 0%	0.085 / 0%
grid	s5	0.995 / 0%	0.120 / 0%	0.210 / 0%
graph	s1	0.733 / 57%	0.025 / 0%	0.115 / 0%
graph	s2	0.981 / 20%	0.050 / 0%	0.125 / 0%
graph	s3	0.722 / 55%	0.090 / 0%	0.100 / 0%
graph	s4	0.688 / 44%	0.125 / 0%	0.120 / 0%
graph	s5	0.935 / 23%	0.035 / 0%	0.115 / 0%

Seed-level means (bootstrap 95%, unit = seed)

- grid vs B-RND: mean 0.981 [0.964, 0.994] (seeds: 0.995, 0.975, 0.990, 0.949, 0.995)
- grid vs B-HEU: mean 0.129 [0.098, 0.165] (seeds: 0.150, 0.080, 0.190, 0.105, 0.120)
- grid vs B-MCTS: mean 0.124 [0.087, 0.168] (seeds: 0.125, 0.125, 0.075, 0.085, 0.210)
- graph vs B-RND: mean 0.812 [0.710, 0.920] (seeds: 0.733, 0.981, 0.722, 0.688, 0.935)
- graph vs B-HEU: mean 0.065 [0.034, 0.100] (seeds: 0.025, 0.050, 0.090, 0.125, 0.035)
- graph vs B-MCTS: mean 0.115 [0.107, 0.121] (seeds: 0.115, 0.125, 0.100, 0.120, 0.115)

H6 results — same-wallclock reading

Score = mean over non-truncated games (win 1 / draw 0.5 / loss 0); trunc = truncation rate; sens = truncations scored 0.5.

Arm	Seed	B-RND score / trunc	B-HEU score / trunc	B-MCTS score / trunc
grid	s1	0.995 / 0%	0.150 / 0%	0.125 / 0%
grid	s2	0.975 / 1%	0.080 / 0%	0.125 / 0%
grid	s3	0.939 / 1%	0.145 / 0%	0.080 / 0%
grid	s4	0.949 / 11%	0.105 / 0%	0.085 / 0%
grid	s5	0.995 / 0%	0.120 / 0%	0.210 / 0%

Arm	Seed	B-RND score / trunc	B-HEU score / trunc	B-MCTS score / trunc
graph	s1	0.798 / 58%	0.045 / 0%	0.075 / 0%
graph	s2	0.926 / 39%	0.055 / 0%	0.110 / 0%
graph	s3	0.713 / 53%	0.060 / 0%	0.105 / 0%
graph	s4	0.631 / 39%	0.100 / 0%	0.150 / 0%
graph	s5	0.980 / 24%	0.045 / 0%	0.095 / 0%

Seed-level means (bootstrap 95%, unit = seed)

- grid vs B-RND: mean 0.971 [0.950, 0.991] (seeds: 0.995, 0.975, 0.939, 0.949, 0.995)
- grid vs B-HEU: mean 0.120 [0.098, 0.142] (seeds: 0.150, 0.080, 0.145, 0.105, 0.120)
- grid vs B-MCTS: mean 0.125 [0.090, 0.168] (seeds: 0.125, 0.125, 0.080, 0.085, 0.210)
- graph vs B-RND: mean 0.810 [0.697, 0.922] (seeds: 0.798, 0.926, 0.713, 0.631, 0.980)
- graph vs B-HEU: mean 0.061 [0.047, 0.081] (seeds: 0.045, 0.055, 0.060, 0.100, 0.045)
- graph vs B-MCTS: mean 0.107 [0.087, 0.130] (seeds: 0.075, 0.110, 0.105, 0.150, 0.095)

Arm contrast — same-examples

Graph – grid difference of seed-level means; bootstrap 95% over seeds (5 per arm, independent).

- vs B-RND: graph–grid = -0.169 [-0.272, -0.062]
- vs B-HEU: graph–grid = -0.064 [-0.111, -0.017]
- vs B-MCTS: graph–grid = -0.009 [-0.055, +0.028]

Arm contrast — same-wallclock

Graph – grid difference of seed-level means; bootstrap 95% over seeds (5 per arm, independent).

- vs B-RND: graph–grid = -0.161 [-0.278, -0.048]
- vs B-HEU: graph–grid = -0.059 [-0.087, -0.029]
- vs B-MCTS: graph–grid = -0.018 [-0.066, +0.025]

H7 ablation results (feeds booklet table H-T3)

Reference: H6 graph full method (journal H6-2026-09-19-comparison-01).

Ablation	Component removed	Outcome
A1 naive adjacency (graph–untyped)	direction-typed edge matrices → one shared matrix	Training diverged (NaN, generation 0, 3/3 seeds) — untrainable at parity settings; eval tables are artifacts of a NaN policy and are excluded as scores (journal H7-2026-09-23-a1-divergence-01). Attribution: typed edges contribute at minimum optimization stability.
A2 no global pooling (graph–nogpool)	global-pooling bias in all layers	NULL effect: diff vs full graph -0.001 [-0.170, +0.165] (B-RND), -0.007 [-0.043, +0.030] (B-HEU), -0.013 [-0.043, +0.013]

Ablation	Component removed	Outcome
		(B-MCTS); seed variance dominates; failure modes unchanged (journal H7-2026-10-02-a2-nogpool-01). Pooling is dispensable at this scale.

A1' supplement (graph-untyped-clip, two-component, D-030) | untyped edges + grad-clip 1.0 | **Trains finite; scores within the full arm's band** — diffs vs full graph: B-RND -0.066 $[-0.254, +0.130]$, B-HEU $+0.032$ $[-0.011, +0.078]$, B-MCTS $+0.008$ $[-0.040, +0.063]$. NEVER attributed to typing alone (clip confound). With A1: typed edges' measurable contribution at this scale is concentrated in optimization stability (journal H7-2026-10-09-a1prime-01). |

Score / cost table (H6 task 7; fig2)

Metric	Grid arm	Graph arm
Parameters	1.44 M	1.47 M (+1.5%)
Best-provider inference (b1)	2.62 ms (CoreML)	3.67 ms (CPU)
Mean run wall-clock (10 gens × 500 games)	18.0 h	36.7 h (2.0×)
Mean self-play cost	13.0 s/game	26.4 s/game
Training throughput benchmark (MPS, batch 128, fwd+bwd)	274 pos/s	138 pos/s
Population score, same-examples	0.411	0.331
Population score, same-wall-clock (T*=18.77 h)	0.405	0.326

Population score = mean over the three frozen opponents of the seed-mean score (truncations excluded, reported separately in the results tables). Wall-clock = self-play generation + training per run (evaluation games excluded), mean over the five seeds; self-play cost = that wall-clock / 5,000 games. Sources: results-*.csv, wallclock.json per run, comparison-controls.md measurements (journal H5-2026-09-10-encoders-01); journal H6-2026-09-19-comparison-01.

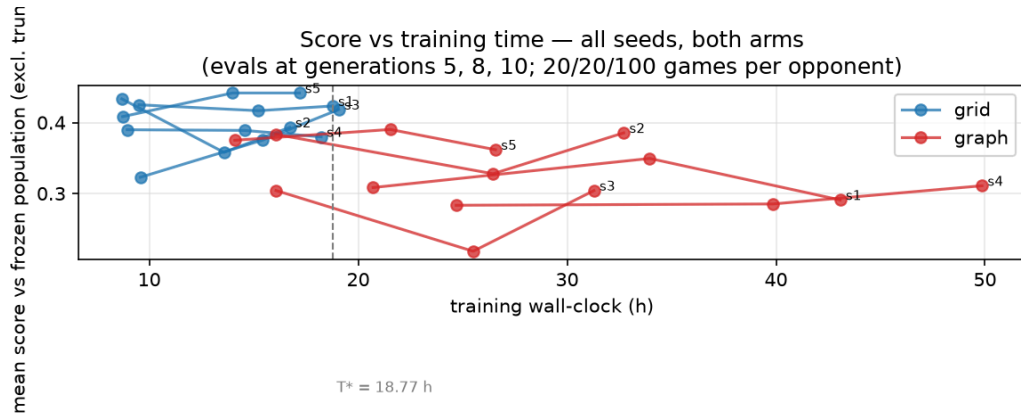


Fig. 1 — Mean score against the frozen population vs training wall-clock, all seeds, both arms (evaluations at generations 5, 8, 10; dashed line = T^*).

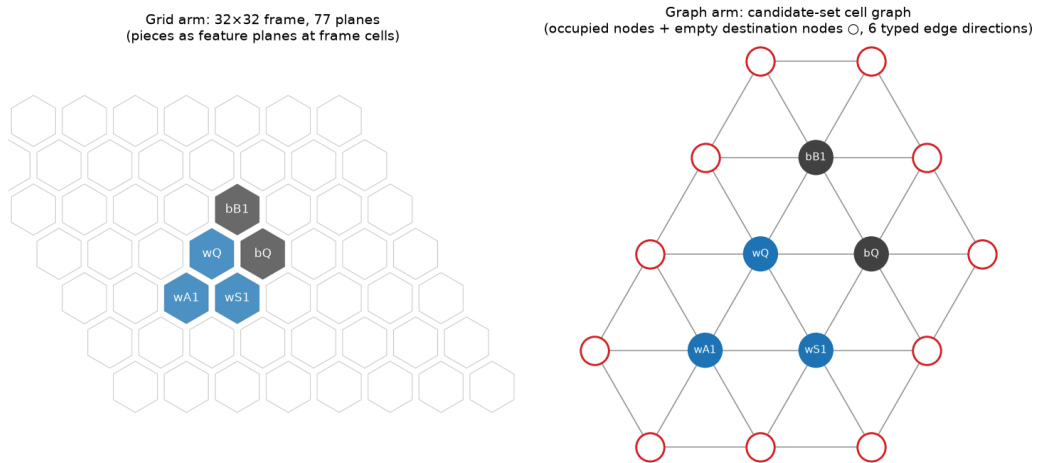


Fig. 3 — The two encodings of one position: grid planes in the 32x32 frame (left) and the cell graph with its six direction-typed relations (right).

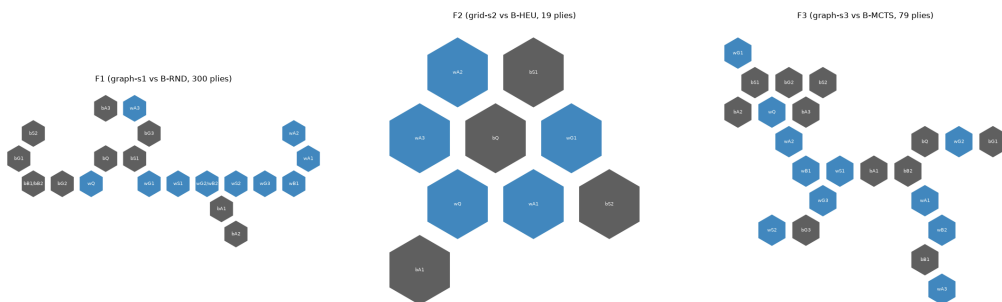


Fig. 4 — Three commented failure positions F1–F3 (details and reproduction status below).

Three commented failure positions (H6 task 7)

Selection criteria are mechanical, stated in `scripts/make_figures.py::fig4`, and applied over the raw per-game CSV records; each selected game is reproduced deterministically (fresh per-game engine seeds) and verified against its CSV row.

F1: graph vs B-RND — longest truncated game (a won position it cannot close: wins material, then shuffles to the 300-ply cap)

- game: graph-s1 vs B-RND, opening 2, A white, 300 plies
- reproduction verified against CSV row: YES
- GameString: Base;InProgress;White[151];wG1;bS1 \wG1;wQ /wG1;bQ -bS1;wS1 wG1-;bG1 -bQ;wG2 wS1-;bB1 bQ/;wS2 wG2-;bA1 -bG1;wG3 wS2-;bS2 \bA1;wA1 wG3\;bA2 bB1-;wB1 wA1\;bB1 bS1;wB2 wB1-;bB1 wG1;wA2 wB2\;bA2 bS2-;wA3 wB2/;bG2 \bS2;wA2 wA3/;bB2 bG2/;wA2 -bG2;bA1 /bQ;wA3 -wA2;bG3 bS1/;wA3 wB2\;bA3 \bG3;wA3 wB2/;bA1 /wS1;wA3 \bA3;bA1 wA3-;wB2 wA1-;bA1 wS2/;wB1 /wA1;bA1 bB2-;wA2 wA3/;bA1 \bB2;wA2 -bA1;bB1 bS1;wA2 wG3/;bA1 bB2\;wB1 wA1;bA1 /bG1;wB2 wB1;bA1 wG2/;wB2 wB1\;bA1 bG3-;wB2 /wB1;bA1 bG3\;wB2 -wB1;bB2 \bG2;wB...

F2: grid vs B-HEU — shortest decided loss (the heuristic’s queen-targeting tactics strike before the net consolidates)

- game: grid-s2 vs B-HEU, opening 2, A black, 19 plies
- reproduction verified against CSV row: YES
- GameString: Base;WhiteWins;Black[10];wG1;bS1 \wG1;wQ /wG1;bQ -bS1;wA1 wG1-;bA1 bS1/;wA1 \bA1;bQ -wG1;wQ /bQ;bS2 -bS1;wA2 \wA1;bS2 /wQ;wA2 -bS1;bS2 wG1\;wA1 -bS2;bA1 /wA1;wA3 \wA2;bA1 /wQ;wA3 -bQ

F3: graph vs B-MCTS — longest drawn game (avoids losing without ever generating winning threats)

- game: graph-s3 vs B-MCTS, opening 19, A white, 79 plies
- reproduction verified against CSV row: YES
- GameString: Base;Draw;Black[40];wS1;bA1 wS1-;wA1 \wS1;bB1 bA1-;wA2 -wA1;bA2 /bB1;wQ \wA2;bQ bB1-;wQ wA2/;bA2 /wA2;wQ \wA2;bA2 -wQ;wB1 -wS1;bA3 bA1/;wA1 bQ-;bA3 bA2/;wA1 /bA1;bG1 bA1/;wA1 bQ-;bG2 -bA3;wA1 -bG2;bG3 /bB1;wA1 /bQ;bB1 wA1;wB2 /wS1;bG2 bA3-;wB2 wS1;bB2 -bQ;wB2 /wS1;bA3 bG2\;wB2 -bG3;bQ bB1-;wB2 /bG3;bS1 bG2-;wB2 /bB1;bS1 -bG2;wB2 /bQ;bQ bB2-;wB2 /bB1;bS2 bG2-;wB2 bB1\;bB1 wB2;wG1 /wS1;bQ bG1-;wG1 \bS1;bS2 wG1/;wA3 -wG1;bS2 -wA3;wG2 /wS1;bG2 -wA2;wG2 -bG1;bG2 bS1-;wG2 bQ-;bG1 wG2-;wG3 /wS1;bG3 wS1...

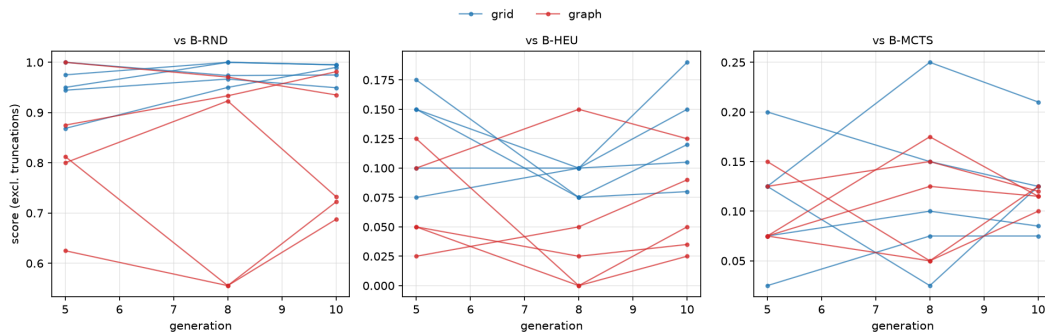


Fig. 5 — Per-opponent score trajectories (mean over 5 seeds, min–max band; final checkpoints per generation; 100 games/opponent at 400 simulations).

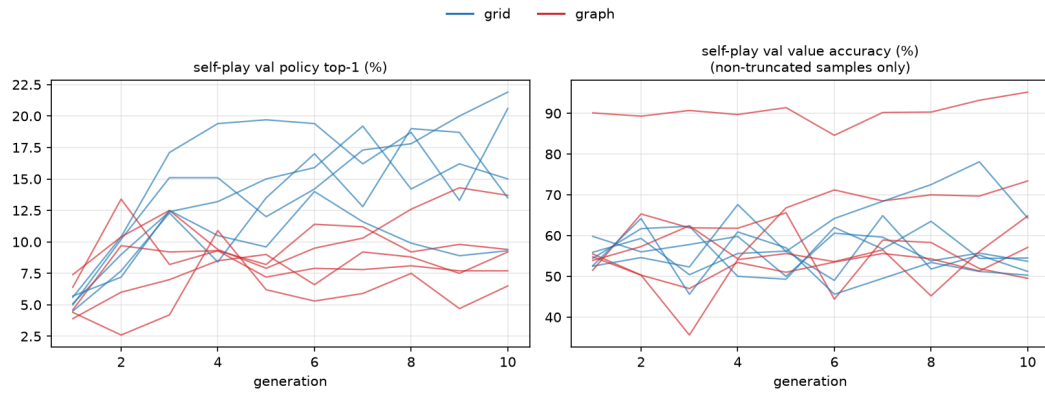


Fig. 6 — Training metrics by generation (epoch-1 policy top-1 and value accuracy, 10 main-campaign runs; ablation runs excluded).

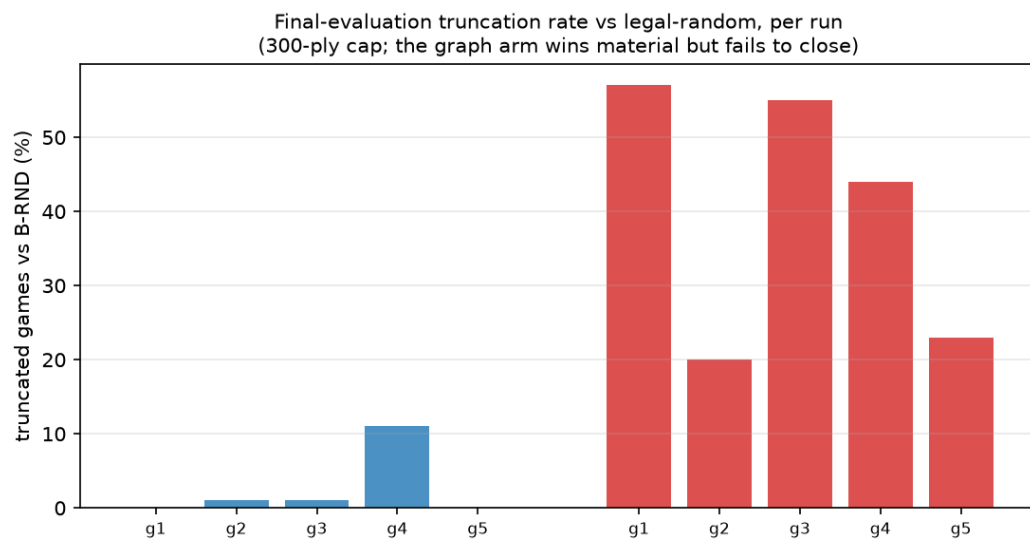


Fig. 7 — Final-evaluation truncation rate against legal-random, per run (final checkpoints, 5 seeds per arm; truncation reported separately from draws per invariant 7).